What Really Predicts a Hit? Artist History, Era Context and Shortcut Features in Hit Song Prediction

Lior Shalon, Nathanael Uzzan
Technion – Israel Institute of Technology, Haifa, Israel
Project in Artificial Intelligence

***Abstract*—Can a computer tell, on the day a song is released, whether it will enter the Billboard Hot 100, the weekly ranking of the 100 most popular songs in the United States? Published studies report 82–89 % accuracy from Spotify audio descriptors, but on different datasets, usually on a single train/test split, and without checking whether the datasets themselves give the answer away. We build a simple, fully described pipeline: 13 audio descriptors, the song's decade, and the artist's *history* (how many Hot 100 songs the artist placed in earlier decades), fed to a Random Forest, and we test every gain with cross-validation (repeated wherever affordable), a corrected t-test and a Bonferroni correction (a few robustness checks use single splits and McNemar's test, and two descriptive comparisons are left untested, as stated where they appear). On the 40,560-song Spotify Hit Predictor dataset, accuracy rises from 0.782 (audio only) to 0.832; the only earlier model on that dataset improves on 50 of 50 random splits when the feature is added. Because that dataset, by its construction rule, never uses the artists of a decade's hits as that decade's negatives, we validate on ISMIR 2019, where a year-matched test gives the transferable estimate: +2.6 points. An audit of every other input we built shows which gains are not real: song titles, spectral descriptors and release year score well only because of how the datasets were assembled. Lyrics are the one content input whose gain survives year matching, an English-only test and unseen artists (+4.4 points), and once a song is out, its first three weeks on the chart predict a top-10 peak with an AUC of 0.89. All details are in the appendix (from p. 16); all code and data are public.**

***Index Terms*—hit song prediction, music information retrieval, feature engineering, data leakage, shortcut learning, reproducibility**

I. Introduction

Whether a song will become a hit is a question with obvious commercial value and a long, contested research history. Early work concluded that "hit song science is not yet a science" [1], later work argued the opposite on other data [2], and the field has since moved from hand-crafted audio descriptors to convolutional networks on spectrograms [3] and multimodal models combining audio, lyrics and metadata [4]–[6].

A large share of recent studies relies on the audio descriptors exposed by the Spotify Web API and on labels derived from the Billboard Hot 100 chart [7]–[10]. Three weaknesses make these results hard to interpret. First, papers report accuracies on *different* datasets, so numbers such as 86 %, 88 % or 89 % are not comparable. Second, gains are usually measured on a single train/test split without a significance test. Third, the datasets are assembled with sampling rules that a model can exploit: a feature may separate hits from non-hits because of how the negative class was chosen rather than because of anything that makes a song successful, an instance of data leakage [11] and shortcut learning [12].

This paper studies two cheap, non-audio signals—the artist's prior chart history and the song's era—under an evaluation protocol designed to address these three weaknesses. Artist history is not a new idea. Interiano et al. [13] add a "superstar" indicator (whether the artist charted in the recent past) to acoustic features of 500,000 UK releases and find that it raises Random-Forest accuracy, and Middlebrook and Sheik [7] list an artist past-performance field among chart-derived fields, although the inputs of their final model are not stated. Neither study tests the gain for significance, compares it with prior models on the same data, or asks whether it reflects the way the negative class was sampled. What we add is a version restricted to strictly earlier periods, a significance-tested measure of its gain on the same data as prior work, and a check on an independent dataset of whether that gain is a genuine signal or a sampling artefact. Our contributions are:

1) A leakage-free formulation of *artist history* which, together with era, improves every classifier we tested on the Spotify Hit Predictor benchmark (Sec. V-A).
2) A *head-to-head* comparison with the only academic work on that benchmark, using its exact protocol, in which our feature improves its models on 50 out of 50 random splits (Sec. V-C).
3) An *external validation* on the peer-reviewed ISMIR 2019 dataset, including a year-matched control that separates a genuine signal from a sampling artefact, and in which our feature exceeds the best published accuracy on the original protocol (protocol A, with a classifier instead of their regression and our own draw of negatives; Sec. V-D).
4) A *robustness* analysis showing that the gain is

not the model recognising artists it was trained on (artist-grouped cross-validation), together with a cold-start breakdown showing that, on the Hit Predictor, artist history does not help artists with no earlier hit and only era does (on ISMIR with year-matched negatives there is a hint that an empty history is informative, not significant after Bonferroni; Sec. V-E).

5) An *ablation* of the final model and a model trained on every input at once, showing where the accuracy comes from (Sec. V-B).
6) An *audit* showing which apparent gains do not survive: title-based features that encode dataset construction, and a rich audio representation whose single-split gain disappears under repeated cross-validation (Appendix J), and a genre control showing that a significant spectral-feature gain on the whole benchmark is a genre shortcut (Appendix J).
7) *Lyrics*: on ISMIR 2019 with year-matched negatives, the bag of words of a song's lyrics adds +4.4 to +5.1 points (+4.4 in the strictest test; with random negatives +2.7 to +4.7, indicative only) and survives year matching, an English-only control and unseen artists, the only content input to do so (Sec. V-G).
8) A *time-series* analysis of the Hot 100: components and forecasting of chart dynamics, market-trend features before release, and early chart trajectories after release (Sec. V-H).

Table I sets each contribution against the closest earlier result.

II. Related Work

Hit prediction. Pachet and Roy [1] found that audio could not predict popularity, while later studies reported above-chance results on chart data [2], [15], with convolutional networks [3] or multimodal models of audio, lyrics and metadata [4]–[6]. Studies built on Spotify descriptors and Billboard labels report 86–89 % [7]–[9], each on its own dataset; the only academic study on the public Spotify Hit Predictor dataset [16] is Du's thesis [10] (84 % and 82 %), and the best result on the peer-reviewed ISMIR 2019 data (released at the 2019 conference of the International Society for Music Information Retrieval) is 75.04 % [4]. **Artist reputation.** Interiano et al. [13] showed that a "superstar" indicator helps predict UK chart success, and Salganik et al. [17] that success feeds on itself. **Evaluation.** Single-split comparisons need McNemar's test [18]; repeated cross-validation needs a corrected t-test [19]. Models can exploit how a dataset was built rather than the task (leakage [11], shortcut learning [12]); in music, Sturm [20] and Flexer [21] showed this for genre recognition. A longer review is in Appendix C.

III. Data

Spotify Hit Predictor (main dataset). Six files, one per decade (1960s to 2010s), each listing songs with 13 Spotify audio descriptors, the artist, the title and the answer: hit (the song entered the weekly Hot 100 in that decade) or non-hit (we also say *flop*). Each file is balanced (about half hits). The documentation states how the non-hits were chosen: songs whose *artist* had no hit in that decade, from "non-mainstream" genres. This rule makes the task easier than in reality, and we return to it throughout. **ISMIR 2019**: Hot 100 hits and non-hits drawn at random from the Million Song Dataset, with 592 audio descriptors computed by Essentia (an open-source audio analysis library); release year and artist come from the Million Song Dataset metadata; its negatives are not chosen by artist or genre, which makes it our honest check. **SpotGenTrack**: 101,909 tracks labelled by Spotify popularity, a 0–100 score (hit if $\geq$ 55; 19 % hits). We also use the full weekly Hot 100 (1958–2026) to compute artist history, 30-second audio previews downloaded from the music streaming service Deezer for two extra audio representations, and the lyrics that the lyrics provider musiXmatch released for the Million Song Dataset. Details are in Appendix D.

IV. Method

A. Overview: what goes in, what comes out

Fig. 1 summarises the whole method. The task is binary classification: *input*, one song described by 20 numbers that are all known on the day it is released (13 audio descriptors, six decade indicators and the artist's history); *output*, the probability that the song enters the Billboard Hot 100, turned into a yes/no answer at 0.5. The model is a Random Forest, an ensemble of decision trees that each ask a sequence of threshold questions about the inputs ("is energy above 0.7?", "did the artist have an earlier hit?") and then vote. The same pipeline is run on three datasets, each with its own audio descriptors. Every claimed gain is measured as the difference in held-out accuracy between the pipeline with and without an input, under the protocol in the middle of the figure. A second, separate pipeline (bottom of the figure) answers a different question that can only be asked after release.

Fig. 2 opens the box: how the forest is trained, and one real song followed from its raw data to the final answer.

B. Features

Audio. The 13 Spotify descriptors (Hit Predictor), the 592 Essentia descriptors (ISMIR) or the same 13 Spotify descriptors (SpotGenTrack; Appendix A) form the baseline of each dataset.

Decade (era). The Hit Predictor comes as six files, `dataset-of-60s.csv` to `dataset-of-10s.csv`; a song's decade is simply the file it comes from: for a hit, the decade in which it was on the Hot 100; for a non-hit, the decade the dataset's author assigned it to (the documentation does not say more; Appendix D). A decade is a category, not a quantity, so we turn it into six yes/no inputs (one-hot encoding: `is_60s`, ..., `is_10s`), exactly one of which

TABLE I

CONTRIBUTIONS: HOW EACH ONE IS OBTAINED, ON WHICH EXPERIMENT, AND HOW THE RESULT COMPARES WITH THE CLOSEST EARLIER WORK. ACCURACY UNLESS STATED; EARLIER NUMBERS ARE AS PUBLISHED. TERMS USED HERE ARE DEFINED LATER: "+5.0" ETC. ARE ACCURACY POINTS (SEC. IV-E); 5×5 CV AND p (SEC. IV-D, IV-E); ISMIR PROTOCOLS A (RANDOM NEGATIVES) AND B (YEAR-MATCHED NEGATIVES) (SEC. V-D); LIBROSA AND FFT, TWO AUDIO REPRESENTATIONS OF A 30-SECOND PREVIEW (APPENDIX A).

Contribution	How we do it	Experiment	Before	Ours
Artist history + era (Sec. V-A)	count of the artist's Hot 100 songs in strictly earlier decades; decade one-hot	Hit Predictor, 40,560 songs, 5×5 CV (5-fold for the 5 classifiers)	superstar indicator, no significance test [13]; 13 descriptors: 0.782	0.832 (+5.0, 25/25 folds, $p = 7{\cdot}10^{-17}$); +4.6 to +5.7 for every classifier
Head-to-head (Sec. V-C)	re-implement Du's protocol, add our feature to his models	2000s and 2010s, 50 random 70/30 splits	0.84 (00s), 0.82 (10s) [10]	0.867 and 0.861; better on 50/50 splits
External validation (Sec. V-D)	same recipe on another dataset, with year-matched negatives	ISMIR 2019, 11,662 songs, 5×2 CV	0.750, wide-and-deep, audio + year [4]	0.763; fair test (year-matched) +2.6, $p = 3{\cdot}10^{-5}$
Unseen and new artists (Sec. V-E)	artist-grouped CV; split test songs by earlier hit or not	Hit Predictor; ISMIR A and B	not tested	gain kept for unseen artists (+5.0, +2.9); new artists: era only on the Hit Predictor (+2.2); ISMIR B artist history +1.4, indicative ($p = .002$)
Ablation (Sec. V-B)	remove one input at a time; shuffle one input; train on everything	Hit Predictor, 5×5 CV	not reported	artist −3.0, decade −2.2, instrumentalness −1.6; everything: +5.8, of which −4.0 is the title artefact
Audit of shortcuts (Appendix J)	CV instead of one split; genre- and year-matched controls	titles, librosa, FFT, year	gains reported on one split	title +6.7 = artefact; librosa +3.8 → +0.9 n.s.; FFT +1.4 → +0.1
Lyrics (Sec. V-G)	musiXmatch bag of words + 8 lexical features, songs with lyrics only	ISMIR 2019, year-matched, English, unseen artists	lyrics in multimodal models on other data [6], [9]	+4.4 ($p = 3{\cdot}10^{-5}$) with year-matched negatives, English only and unseen artists
Early trajectories (Sec. V-H)	first k weekly positions + climb, Random Forest	31,925 Hot 100 songs, top-10 peak	early views predict later views of online content [14]	AUC 0.892 with 3 weeks + context (0.593 with context only)

TABLE II

DATASETS. "HIT" DEFINITIONS DIFFER; ACCURACY IS ONLY COMPARABLE WITHIN A ROW.

Dataset	Songs	Hits	Label / negatives
Spotify Hit Predictor [16]	40,560	49.4 %	Hot 100 in decade / rule-based
audio subset (Deezer clips)	3,562	52.0 %	same
ISMIR 2019 [4] (balanced)	11,662	50.0 %	Hot 100 / random Million Song Dataset
SpotGenTrack [5]	101,909	19.4 %	popularity ≥ 55

equals 1 (Fig. 2, step 2b). Because every file is balanced, the decade alone says nothing about whether a song is a hit (0.503 accuracy, at chance level: always answering "non-hit" gives 0.506). It helps because *what a hit sounds like changes with the era* (Fig. 3): a 1990s hit is louder than a 1960s hit, and the median 2000s non-hit is louder than the median hit of any decade from the 1960s to the 1980s; in the 1960s–90s hits are less acoustic than non-hits, in the 2000s slightly more; in the 1970s–90s hits are longer than non-hits, in the 2000s–10s shorter. With the decade as input, the trees can learn a different rule for each era. ISMIR and SpotGenTrack have no decade files; there the era is one number, the release year on ISMIR (e.g. 1987) and the release decade on SpotGenTrack (e.g. 1990, from the album release date). On ISMIR we also test the decade, the year rounded down to ten.

Artist history. Let $a(s)$ be the normalised primary artist of song s (on the Hit Predictor and for the chart history: lower-cased, accents removed, cut at the first collaborator separator such as "feat.", "&", "with" or a comma, each written with a space on both sides (full list in Appendix A), then punctuation removed; on ISMIR and SpotGenTrack, whose artist fields are cleaner: only lower-cased and trimmed, on ISMIR the whole artist field, on SpotGenTrack the first artist of its artist list) and $t(s)$ its period (decade on the Hit Predictor, release year elsewhere). With $\mathcal{H}$ the set of reference hits, each element s' being one (artist, title, period) entry, so that a title charting in two decades gives two entries,

$$h(s) = \ln\bigl(1+\bigl|\{\, s' \in \mathcal{H} : a(s') = a(s),\ t(s') < t(s) \,\}\bigr|\bigr). \quad (1)$$

The strict inequality guarantees that no information from the song's own period or from the future is used. (Accuracy differences such as "+5.0 pts" are in points: 0.01 of accuracy; Sec. IV-E.) One subtlety remains: a hit whose chart run crosses a decade boundary, or that re-entered the chart, also appears among the artist's titles of the previous decade, and so counts itself. This concerns 219 songs (211 hits, 0.5 %); excluding the song's own title from the count leaves the result essentially unchanged (0.831 instead of 0.832; still +5.0 pts over audio alone, 25/25 folds, $p = 4\times10^{-18}$; `r8_future_work.py selfcount`; this check was run on the Hit Predictor only), so we keep the simpler definition. In words: for every song, we look up

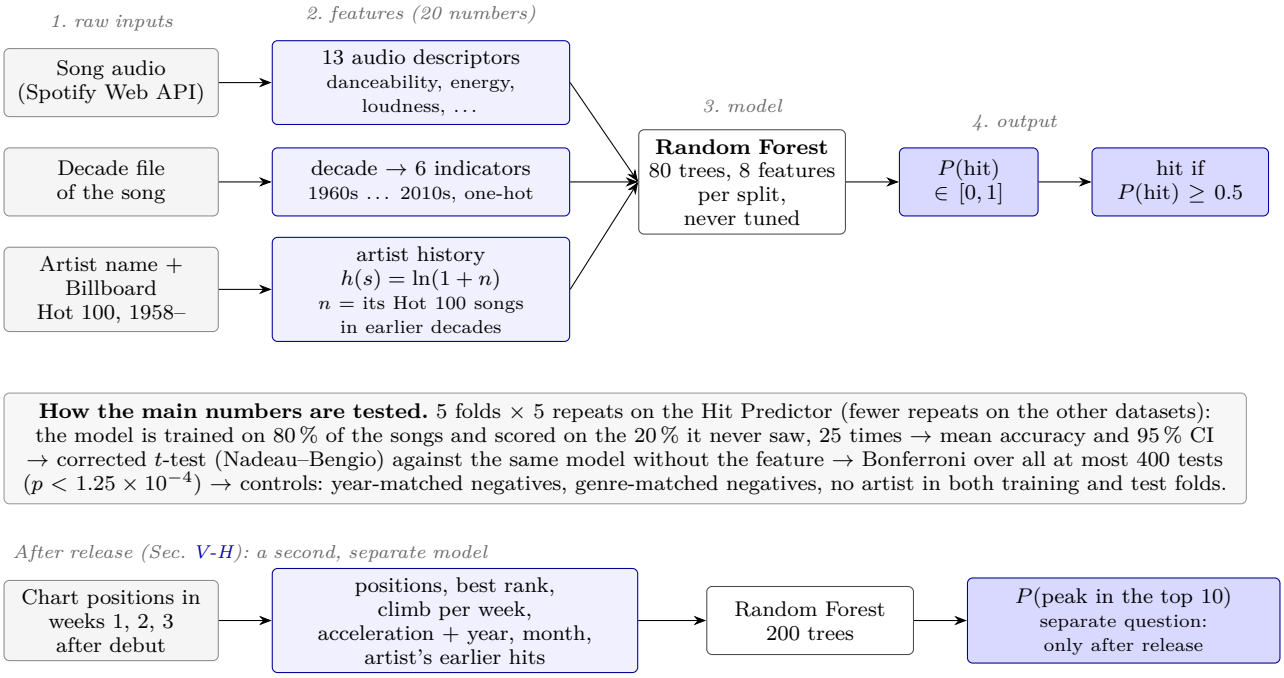


Fig. 1. The prediction pipeline. Top: the final model predicts, on release day, whether a song will enter the Hot 100, from 20 numbers per song. Middle: the protocol under which every gain in this paper is measured. Bottom: a separate model, used only once a song has charted, predicts whether it will reach the top 10 from its first three weeks.

its artist in the full Hot 100 history and, for each decade before the song's own decade, count how many different songs that artist had on the chart in that decade, then add these counts (a song that charted in two decades counts once in each); the natural logarithm ln compresses large counts (an artist with 40 earlier hits is not twice as likely to chart again as one with 20). Fig. 4 gives a worked example. The value is computed separately for every artist and every period, so the same artist gets a larger value in each later decade, and an artist with no earlier hit always gets $h(s) = 0$.

On the Hit Predictor, $\mathcal{H}$ is the full weekly Hot 100 history (distinct titles per artist and decade, the decade being that of the chart week); on ISMIR and SpotGenTrack it is the set of positive songs of the dataset itself, counted by earlier release year. On the Hit Predictor, 52.1 % of songs have an artist that appears somewhere in the Hot 100 history: 99.99 % of the hits and 5.4 % of the flops. The unmatched songs are therefore flops by artists who never charted, not matching failures (Appendix J).

C. Models

The main model is the Random Forest [22] configuration of Middlebrook and Sheik [7] (80 trees, 8 candidate features per split, or all of them for a block with fewer than 8 inputs; splits chosen by Gini impurity, Fig. 2), kept frozen throughout: we do not tune hyperparameters, so that gains come from the features and remain comparable with the published configuration. Robustness checks use XGBoost [23], logistic regression, k-NN and an RBF support vector machine (each defined in Appendix P). External datasets use a Random Forest with 200 trees and $\sqrt{d}$ candidate features per split (d = number of inputs), the usual default when d is large (592 descriptors on ISMIR). All models are the scikit-learn 1.8 and XGBoost 3.2 implementations (Python 3.14); audio features use librosa 0.11.

D. Evaluation protocol

Every model is always tested on songs it did not see during training (*held-out* songs). The main comparisons on the Hit Predictor are reported with two estimates; most other experiments use cross-validation only, and a few robustness checks use single splits (stated where they appear).

Cross-validation (the estimate our conclusions rest on). The songs are shuffled and cut into five equal parts (*folds*), each with the same share of hits (*stratified*). The model is trained on four parts (32,448 songs on the Hit Predictor) and tested on the fifth (8,112 songs); this is repeated so that every part is the test part once, giving five accuracies. The whole procedure is then repeated five times with a different shuffle, giving $5 \times 5 = 25$ test accuracies ("5×5 CV"; scikit-learn `RepeatedStratifiedKFold`, seed 42). Two configurations, e.g. with and without artist history, always use the *same* 25 splits, so they can be compared fold by fold. Anything learned from data (standardisation, PCA, the text models) is fitted on the training part only. Two exceptions: the rejected market-trend features (Appendix A), and artist history on ISMIR and SpotGenTrack, which counts the dataset's own earlier hits,

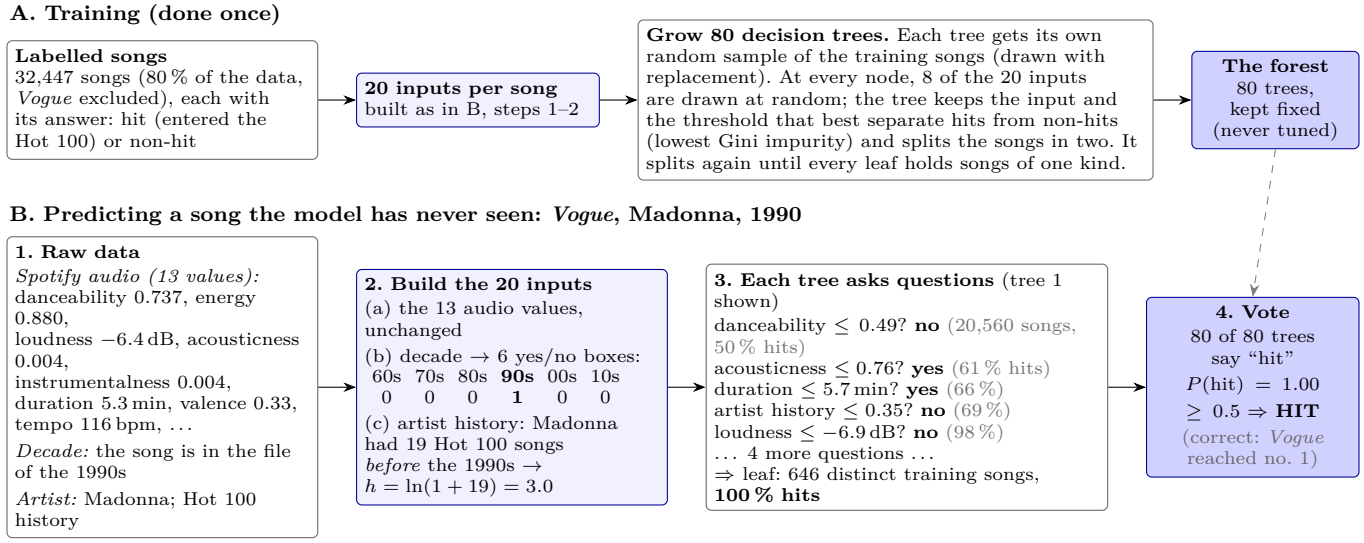


Fig. 2. The algorithm in detail. For this illustration only, *Vogue* is set aside and the other 40,559 songs are split once 80/20 (seed 42): 32,447 for training, 8,112 for testing, plus *Vogue*, so 8,113 test songs. A: the Random Forest is trained once on the labelled songs. B: *Vogue*, never seen in training, followed through the four steps. The percentages in step 3 are the share of hits among the training songs that reach each question in tree 1. The root holds 20,560 songs, not 32,447, because the tree's sample is drawn *with replacement*: it contains 32,447 draws, but some songs are drawn several times and about 37 % not at all, leaving 20,560 distinct songs. *Gini impurity* measures how mixed a group is $(1 - q^2 - (1 - q)^2$ for a hit share q: 0 for a pure group, 0.5 for half and half). Every song is processed in exactly the same way; across the 8,113 test songs, the answer is right 83.3 % of the time.

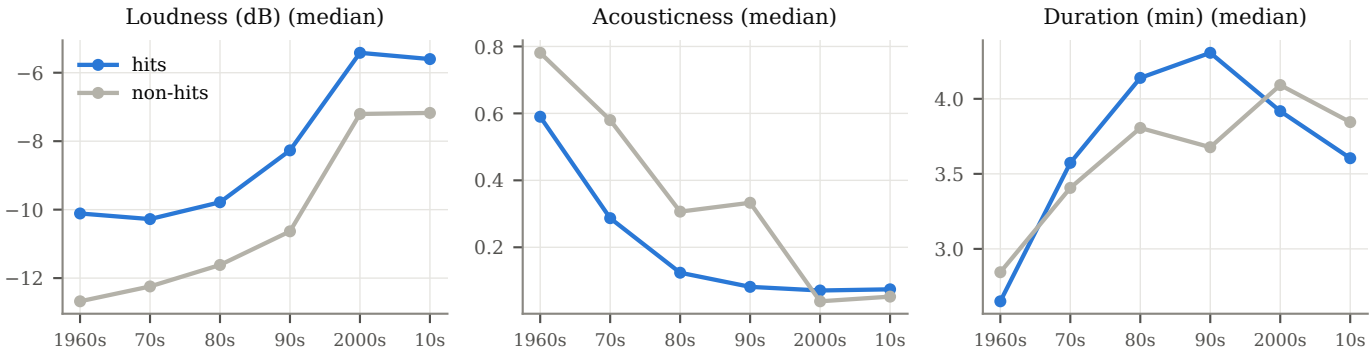


Fig. 3. Why the decade is useful although every decade is balanced: the median value of three audio descriptors for hits (blue) and non-hits (gray) in each decade of the Spotify Hit Predictor. What separates hits from non-hits moves, and sometimes reverses, from one era to the next.

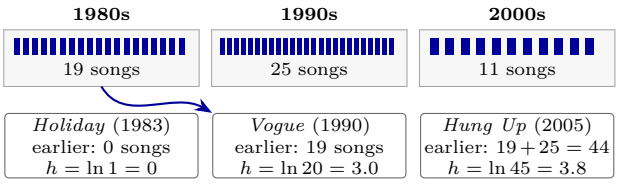


Fig. 4. Artist history, worked example for Madonna. Each tick is one distinct title she had on the Hot 100 in that decade (decade of the chart week). For a song from the 1990s (*Vogue*), only her songs from *earlier* decades are counted (the 19 of the 1980s, arrow), never those of the 1990s themselves, so the value is known on release day. A song from her first hit decade gets 0 (*Holiday*), and a 2000s song counts the 1980s and 1990s (19 + 25 = 44). ln is the natural logarithm.

including songs that fall in the test folds; this uses only information from strictly earlier years, so it is legitimate in time. Where one run is expensive we use fewer repeats: 5×2 on ISMIR (592 descriptors per song) and 5×1 on SpotGenTrack (101,909 songs). Here "5×2" always means 5 folds repeated twice, not the 2-fold "5×2cv" design of Dietterich [18].

Canonical split (a second, independent check). One fixed split, seed 42: from the 40,560 songs, a sample of 24,000 is drawn and divided into 20,000 training, 2,000 validation (not used) and 2,000 test songs by three successive scikit-learn `train_test_split` calls, each stratified by label with seed 42: 24,000 songs kept out of 40,560, then 20,000 of them for training, then the other 4,000 cut into 2,000 validation and 2,000 test songs. This was the split of the first version of the study and we keep it unchanged so that its numbers can be compared over time; it uses fewer training songs, which is why its accuracies are about one

point lower. On ISMIR and SpotGenTrack the canonical split is a stratified 75/25 split.

Out-of-fold text features. For the song title (and later the lyrics), a TF-IDF + logistic-regression model (Appendix P) is trained on the training part only, with an inner stratified, shuffled 5-fold split (seed 42) so that every training song gets a probability from a model that did not see it; the test songs get the probability of the model trained on the whole training part. That probability is then used as one input.

E. Metrics and statistics in plain words

Metrics. *Accuracy* is the share of test songs whose answer (hit or non-hit) is right. A gain in *points* (pts) is a difference in accuracy times 100: going from 0.782 to 0.832 is +5.0 pts. On the hit class, *precision* is the share of songs predicted "hit" that are hits, *recall* the share of hits that are predicted "hit", and *F1* their harmonic mean $2PR/(P+R)$. *ROC-AUC* (area under the receiver operating characteristic curve) uses the predicted probability instead of the yes/no answer: it is the probability that a randomly chosen hit gets a higher score than a randomly chosen non-hit (0.5 = chance, 1 = perfect); it is useful when classes are imbalanced (SpotGenTrack), where always answering "non-hit" already gives high accuracy. *Wins* (e.g. 25/25) is the number of the 25 folds in which the configuration with the new input is more accurate than without it (a tie does not count as a win).

What a p-value says. Suppose the new input were in fact useless. Its measured gain would then be pure noise, sometimes positive, sometimes negative. The p-value is the probability of seeing a gain at least as large in absolute value as the one we measured if the input were useless. A small p (e.g. 10^{-10}) means that such a gain would almost never appear by chance; "n.s." (not significant) means $p \geq 0.05$.

The test we use (corrected resampled t-test [19]). For each of the J folds we compute the paired difference d_j = accuracy with the input − accuracy without it (or the same difference of AUC or F1 when those are tested), then its mean $\bar{d}$ and its sample variance s^2 (here s is a standard deviation, not a song). A naive paired t-test would use $t = \bar{d}/\sqrt{s^2/J}$. But the 25 folds are not independent: any two training sets share most of their songs, so the 25 differences vary less than truly independent estimates would, and the naive test finds "significant" gains far too easily. Nadeau and Bengio correct this by inflating the variance:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s^2}}, \qquad (2)$$

where $n_{\text{test}}/n_{\text{train}}$ is the ratio of test to training songs in one fold (8,112/32,448 = 1/4 in 5-fold CV, 3/7 for 70/30 splits). The p-value is two-sided, from a Student t distribution with $J-1$ degrees of freedom. *Worked example* (Table III, audio vs. audio + decade + artist): over the 25 folds the gain has mean $\bar{d} = 0.0504$ and standard deviation $s = 0.0045$ (smallest 0.041, largest 0.058). The naive test would give $t = 56$; the corrected one gives $t = 0.0504/\sqrt{(1/25 + 1/4) \times 0.0045^2} = 20.8$, and with 24 degrees of freedom $p = 7 \times 10^{-17}$. The per-fold accuracies of every experiment are in the repository (`paper_results/*_folds.csv`), so every p-value can be recomputed.

Confidence intervals. The 95 % interval printed next to an accuracy is the mean of the J fold accuracies $\pm\, t_{0.975,J-1}\, s_{\text{acc}}/\sqrt{J}$, where s_{acc} is the standard deviation of the J fold accuracies and $t_{0.975,J-1}$ the 97.5 % quantile of a Student t distribution with $J-1$ degrees of freedom (2.06 for $J = 25$). It is not corrected for the overlap of folds, so it describes how stable the 25 estimates are and is narrower than the true uncertainty; significance is always judged with the corrected test above, never from the intervals.

Single split: McNemar's test [18]. On one test set, we count b, the songs only the old model gets right, and c, the songs only the new model gets right. If the two models were equally good, b and c would be about equal; the test statistic is $(|b-c|-1)^2/(b+c)$, compared with a χ^2 distribution with one degree of freedom (an exact binomial test when $b + c < 25$).

Many tests at once (Bonferroni). With many tests, some small p-values appear by chance alone: among 100 useless inputs, about five would reach $p < 0.05$. Bonferroni divides 0.05 by the number of tests. We do not rely on an exact count: every p-value printed in the paper that compares two configurations, counting a test again each time it is printed, plus the exploratory tests we ran but do not print (Appendix M), stays well below 400, and we divide by 400, a deliberately generous family size. A result is therefore called significant after Bonferroni only if $p < 0.05/400 = 1.25\times10^{-4}$; results with $1.25\times10^{-4} \leq p < 0.05$ are called *indicative*. Inputs are judged on the accuracy gain unless a row says otherwise (so the SpotGenTrack lyrics stay indicative on accuracy, $p = 0.002$, although their AUC gain, $p = 2 \times 10^{-5}$, would pass).

Notation. A few letters have a second, local meaning, always stated where it is used: p is a p-value, except for an ARIMA order (Appendix O) and a chart position p_w (Appendix A); d is the number of inputs in a model, d_j a fold difference, and the ARIMA differencing order in Appendix O; t is the t statistic, and $t(s)$ the period of song s; k counts folds, neighbours (k-NN), weeks or decades depending on the section.

Rounding. Gains Δ are computed from unrounded accuracies, so a gain can differ by 0.1 pts from the difference of the rounded values printed in a table (e.g. 0.8103 − 0.7817 = 0.0286, printed +2.9, although .810 − .782 = .028).

Two kinds of "new" artist. Two different situations are easy to confuse, and we keep them apart throughout. An *unseen* artist is one whose songs are absent from the

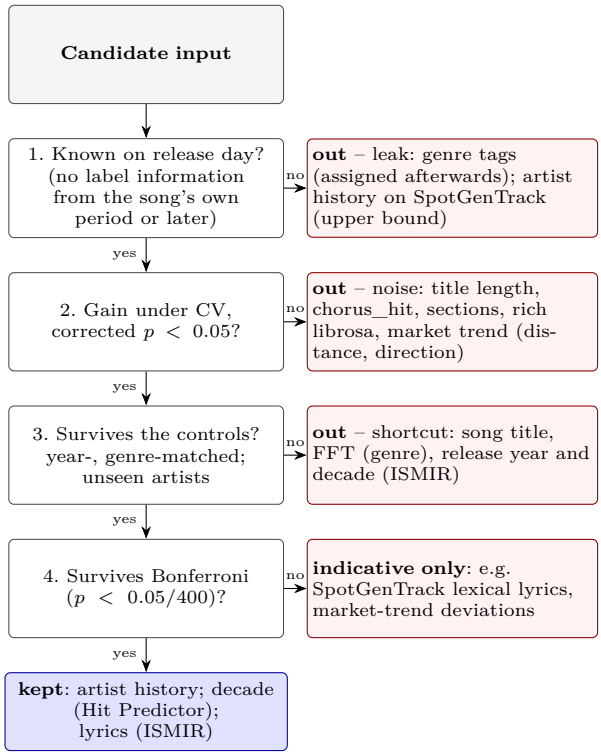


Fig. 5. How every candidate input was kept or rejected. Each input is tested in this order; the boxes on the right list the inputs that stopped at each question. An input *fails* question 3 when, under at least one of the controls we ran on it (each applied separately, or combined where stated), its gain is no longer significant after Bonferroni. The after-release trajectory model answers a different question and is outside this flowchart.

training folds; its chart history before the song is still known, so $h(s)$ can be positive. We test unseen artists with artist-grouped cross-validation (Sec. V-E), to check that the model does not simply recognise artists it was trained on. A *new* (cold-start) artist is one with no earlier hit in the reference set $\mathcal{H}$ (the full Hot 100 on the Hit Predictor; on ISMIR and SpotGenTrack only the dataset's own hits, so an ISMIR "new" artist may have Hot 100 songs that are not in the dataset), so $h(s) = 0$ by definition. Our claims that the gain "holds for unseen artists" never mean that it helps new artists. A value of 0 could still carry information, since the model learns from the known artists what an empty history means. On the Hit Predictor it does not (+0.4 pts, n.s.; only era helps new artists); on ISMIR with year-matched negatives there is a hint that it does (+1.4 pts, $p = 0.002$, above the Bonferroni threshold, so indicative only; Table XIV).

F. How inputs were selected

We did not choose the inputs in advance. Every candidate we built—decade, artist history, song title, title length, `chorus_hit/sections` (two analysis fields of the dataset; Appendix A), a 185-dimensional representation computed with the audio library librosa, 31 "FFT" descriptors (spectral and rhythm statistics of the spectrogram of a 30-second preview), Spotify artist genres, market-trend features and lyrics—went through the same four questions (Fig. 5). We first ask whether the input is known on release day. We then measure its gain under cross-validation (5×5; 5×2 on ISMIR; a single 5-fold run on SpotGenTrack) with the corrected t-test, not on one split. We then re-test it under the controls that remove the known construction shortcuts of each dataset: negatives matched by year (ISMIR), negatives matched by genre (Hit Predictor), and no artist shared between training and test folds. Each control is applied separately (the title also gets a combined genre + formatting control, Sec. V-F); an input *fails* when, under at least one of the controls we ran on it, its gain is no longer significant after Bonferroni. A genre-matched *and* artist-grouped run at once was not done. On ISMIR, protocol A (random negatives) contains the release-year shortcut, so ISMIR inputs are judged on the year-matched protocol B, with the other controls (English only, unseen artists) applied on top of it; protocol-A numbers are reported for comparison with prior work. Finally we require significance after a Bonferroni correction over all the tests in the paper. We call an input that passes all four questions a *surviving* input, and we rank all tested gains by this criterion in Table VI (the *survival ranking*). Artist history survives on the Hit Predictor and on ISMIR; on SpotGenTrack its gain is only an upper bound, because popularity is measured today (Sec. VI). The decade survives on the Hit Predictor only (where it matters through its interactions with audio); on ISMIR, once years are matched, the era adds nothing (Sec. V-D). Bag-of-words lyrics survive on the one dataset where we could attach them to both hits and non-hits, ISMIR (Sec. V-G); on SpotGenTrack only six lexical descriptors are available, and their gain is indicative. We also trained a model on everything at once, and we removed one input at a time from both that model and the final model (Sec. V-B).

TABLE III
SPOTIFY HIT PREDICTOR (40,560 SONGS), FROZEN RANDOM FOREST. CANONICAL SPLIT (2,000 TEST SONGS, MCNEMAR) AND 5×5 CROSS-VALIDATION (MEAN, 95 % CI, CORRECTED t-TEST, FOLDS IMPROVED OUT OF 25).

	Canonical		Cross-validation		
Features	Acc.	p	Acc. [95 % CI]	AUC	p (wins)
Audio (13)	.773	–	.782 [.780,.784]	.859	–
+ decade	.796	$3{\cdot}10^{-4}$	.802 [.800,.804]	.880	10^{-10} (25)
+ artist	.800	$6{\cdot}10^{-4}$	.810 [.808,.813]	.900	10^{-11} (25)
+ decade + artist	**.822**	$2{\cdot}10^{-9}$	**.832 [.830,.834]**	**.918**	$7{\cdot}10^{-17}$ (25)
Decade + artist only	.668	–	.670 [.669,.672]	.685	–

V. RESULTS

A. Main result on the Spotify Hit Predictor

Table III shows that each feature helps on its own and that their gains are nearly additive. The canonical split trains on fewer songs (20,000 instead of 32,448), so its

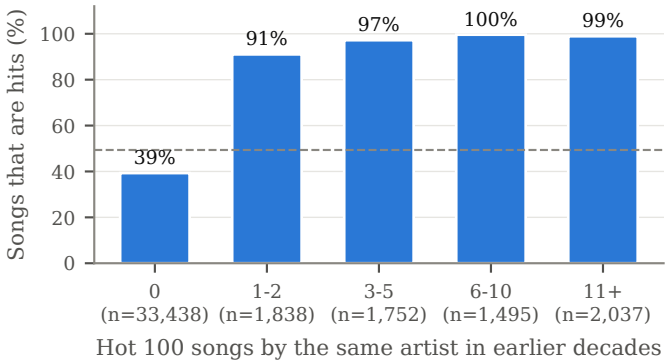


Fig. 6. Share of hits by the number of Hot 100 songs the artist placed in earlier decades (Spotify Hit Predictor, 40,560 songs). Dashed line: overall hit share (49 %).

accuracies are about one point lower, but it ranks the configurations in the same order. The combined gain is +5.04 pts and is positive in all 25 folds. It is not bought by trading recall for precision: on the hit class, precision rises from 0.758 to 0.828 and recall from 0.819 to 0.833, so F1 rises from 0.787 to 0.831 (+4.3 pts, 25/25, $p = 7 \times 10^{-15}$). Without audio the two features reach only 0.670, so they complement rather than replace the descriptors. The relation between artist history and the label is strongly nonlinear (Fig. 6): 39.3 % of the 33,438 songs whose artist had no earlier hit are hits, against 91.0 % for one or two earlier hits and 97.2–99.6 % beyond.

The gain holds in every decade (+3.7 to +6.6 pts), for five very different classifiers (+4.6 to +5.7 pts: Random Forest, XGBoost, logistic regression, k-NN, SVM), and when the model is trained on earlier decades and tested on a later one it has never seen (+1.9 to +3.4 pts). XGBoost is the strongest classifier (0.845) but ties with the frozen Random Forest on the canonical split ($p = 0.87$). Tables and details: Appendix E.

B. Where the accuracy comes from

Which audio descriptors matter. A principal component analysis (PCA: new axes, combinations of the inputs, ordered by the share of variance they explain) of the 13 descriptors (Appendix F, Fig. 11) finds an *intensity* axis (energy, loudness against acousticness) and a *mood* axis (danceability, valence), but the directions of largest variance are not the ones that separate hits, and replacing the descriptors by their components only loses accuracy (−1.7 pts with all 13), so we keep the raw descriptors.

Ablation: remove one input at a time. To see where the power of the final model comes from, we removed each of its inputs in turn and re-ran the 5×5 cross-validation, comparing every reduced model with the full one fold by fold (Fig. 7a). Removing artist history costs the most (−3.0 pts, 0/25 folds better, $p = 7 \times 10^{-13}$), then the decade indicators (−2.2 pts, $p = 5 \times 10^{-13}$), instrumentalness (−1.6 pts, $p = 6 \times 10^{-9}$) and duration (−1.1 pts, $p = 2 \times 10^{-7}$). These four survive the Bonferroni correction. Acousticness, speechiness, energy, danceability and valence each cost 0.3–0.6 pts ($0.004 \leq p \leq 0.04$), because the other descriptors largely carry the same information. Removing loudness (−0.3 pts), key, liveness, time signature, mode or tempo changes nothing ($p \geq 0.30$). Keeping one block alone shows that the blocks are complementary rather than redundant: the 13 descriptors alone reach 0.782, decade and artist history together without audio 0.670, the decade alone 0.503 (chance, since every decade is balanced), and all three together 0.832. Removing all 13 descriptors at once costs −16.2 pts. So audio carries most of the accuracy, and artist history and era add to it.

Permutation importance. Removing an input lets the model re-learn with the others, so correlated descriptors can stand in for one another. As a second view, we shuffle one input at a time in the held-out folds of the trained final model (the 5 folds of the first repeat, 5 shuffles each), which measures how much the model relies on it (Fig. 7b). The ranking is the same at the top: artist history (8.9 pts lost when shuffled), then instrumentalness (6.3), acousticness (4.9), the decade block (4.3; its six indicators are shuffled together), duration (3.9) and danceability (3.4), with key, liveness and time signature near zero. Duration is an instructive case. On its own it barely separates hits (AUC 0.51; Appendix A), but the model relies on it, presumably because song lengths changed across decades and its effect depends on the era. Its value comes from interactions, like that of the decade.

Training on everything at once. A model with every input we built (8 blocks, including song titles, FFT, market trend) reaches 0.883 against 0.825 for the final model on the same songs, but removing the song title alone takes it back to 0.843, and the rest of the gain comes from FFT, which fails the genre control, and from the market-trend features, which carry only the era (Table XI, Appendix F). We therefore keep the 20 inputs of the final model.

C. Head-to-head with prior work on the same benchmark

Du [10] trained one Random Forest (450 trees, $m_{\text{try}} = 2$ candidate features per split, at most 30 leaves) and one logistic regression per decade on the 2000s and 2010s, on 15 inputs (the 13 descriptors plus `chorus_hit` and `sections`), with a single 70/30 split. We also run our own frozen Random Forest under the same protocol (Table XII). Because the original R random seed cannot be replayed, we repeat the split with 50 seeds (stratified 70/30 splits with seeds 0 to 49; each seed also seeds the models), on our de-duplicated decade files (5,855 songs in the 2000s, 6,280 in the 2010s). Our re-implementation lands on or slightly above the published figures (Table XII, Fig. 8), which validates the protocol. Adding artist history to *their* models, with *their* inputs, improves mean accuracy by +2.1 to +3.2 pts depending on the model and decade, the improvement is positive on every one of the 50 splits, and

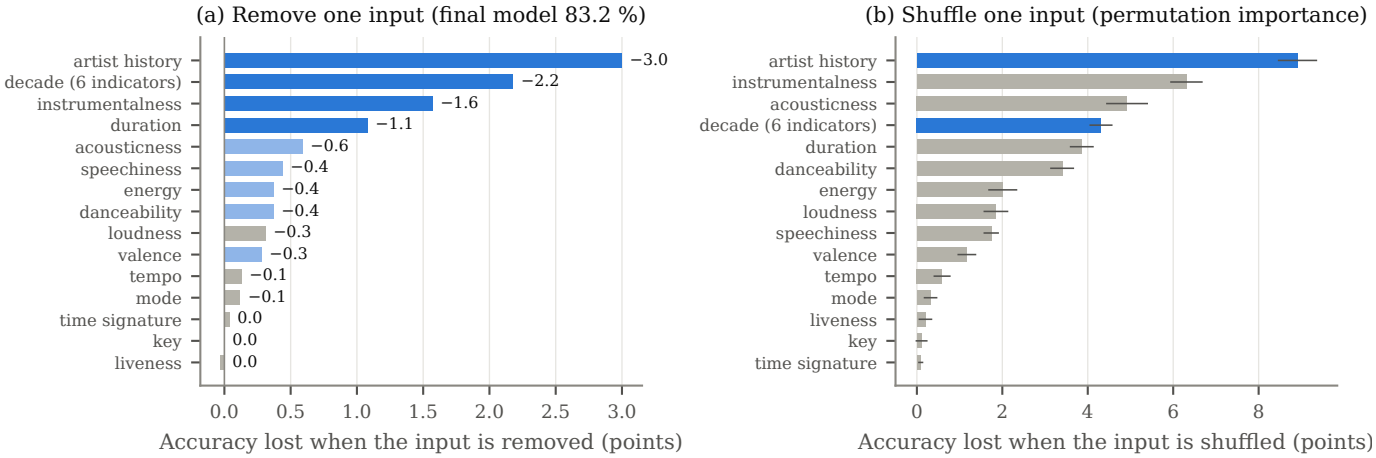


Fig. 7. Where the accuracy of the final model comes from (Hit Predictor, 40,560 songs). (a) Accuracy lost when one input is removed and the model is retrained (5×5 CV; dark blue: significant after Bonferroni, light blue: $p < 0.05$, gray: not significant). (b) Accuracy lost when one input is shuffled in the held-out folds of the trained model (5 folds × 5 shuffles, mean ± s.d.). Blue: non-audio inputs.

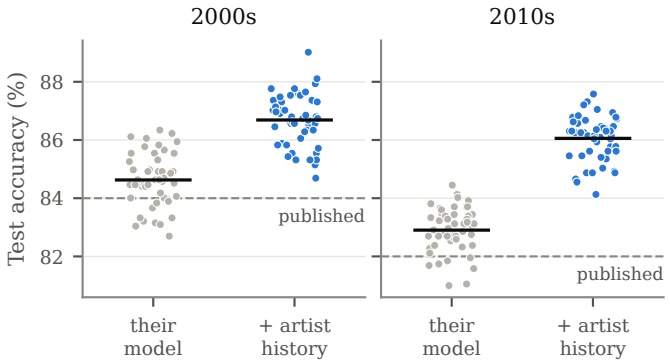


Fig. 8. Head-to-head with Du [10]: accuracy of their Random Forest over 50 random 70/30 splits (dots; bar = mean), without (gray) and with (blue) artist history. Dashed line: their published accuracy.

TABLE IV
ISMIR 2019 DATA, RANDOM FOREST ON 592 ESSENTIA DESCRIPTORS, 5×2 CROSS-VALIDATION. A: BALANCED RANDOM NEGATIVES (AS IN [4]). B: NEGATIVES MATCHED TO HITS YEAR BY YEAR. p: CORRECTED t-TEST VS AUDIO; ALL 10 FOLDS IMPROVE WHERE SIGNIFICANT.

	A: random negatives		B: year-matched	
Features	Acc. [95 % CI]	p	Acc. [95 % CI]	p
Audio (Essentia)	.708 [.700,.717]	–	.682 [.677,.688]	–
+ year	.739 [.734,.744]	$2{\cdot}10^{-4}$	.681 [.674,.688]	.66
+ artist	.733 [.727,.740]	$2{\cdot}10^{-5}$	**.708** [.702,.714]	$3{\cdot}10^{-5}$
+ year + artist	**.763** [.758,.768]	$7{\cdot}10^{-8}$	**.708** [.702,.715]	$3{\cdot}10^{-5}$
Year alone	.702 [.697,.707]	–	.448 [.444,.451]	–

Best published [4]: 75.04 % (wide-and-deep, audio + year, 5-fold CV)

the improved models exceed the published accuracy on 100 % of the splits. The decade feature is constant within a per-decade model and is therefore not used here.

D. External validation

ISMIR 2019. (Table IV, Fig. 12.) With random balanced negatives (protocol A, as in the original study), release year alone reaches 70.2 %—as much as 592 audio descriptors—because hits and non-hits come from different eras. Audio + year, which mirrors the inputs of the best model of Zangerle et al., reaches 73.9 %; adding artist history raises it to 76.3 % (+2.4 pts over audio + year, 10/10 folds, $p = 7\times10^{-6}$), above the published 75.04 %. Protocol B removes the era shortcut by drawing, for every hit, a non-hit from the same release year (in every year we keep as many hits as non-hits, the smaller of the two counts; one draw, seed 42). The non-hit pool is large enough in every year, so all 5,831 hits are kept (11,662 songs). Year and decade then become useless (audio + year 0.681, $p = 0.66$; audio + decade 0.681, $p = 0.56$; in protocol A audio + decade reaches 0.738, $p = 1.1 \times 10^{-4}$, a shortcut listed in Table VI; year alone falls below chance, Appendix Q), while artist history still adds +2.6 pts ($p = 3\times10^{-5}$, 10/10 folds). This is the cleanest estimate of the feature's value: the negatives are not selected by artist and cannot be separated by date.

SpotGenTrack. With popularity as the label and 81 % negatives, accuracy barely moves, but ranking does: artist history and decade raise the ROC-AUC from 0.690 to 0.777 and the share of hits found from 8 % to 25 % (Appendix H).

E. Unseen artists and cold start

Unseen artists. With random folds, songs by the same artist can be both in training and in test, so a model could recognise an artist from the audio. With folds grouped by artist, so that no test artist is ever seen in training, every gain remains: +5.0 pts on the Hit Predictor and +2.9 pts in the year-matched ISMIR test ($p = 9\times10^{-7}$). **New artists.** For songs whose artist had no earlier hit, artist history is

0 and does not help on the Hit Predictor (+0.4 pts, $p = 0.17$); the decade still helps them (+2.2 pts, $p = 2 \times 10^{-9}$). On the Hit Predictor the whole gain of artist history comes from the 18 % of songs by established artists, 97 % of which are hits, which is the dataset effect mentioned above. Table and details: Appendix I.

F. Audit: which gains are not real

Several inputs gave large, significant gains that do not survive a closer look (Table VI). The *song title* adds +6.7 pts, more than any musical input, because the titles of the non-hits are written differently: on the genre-matched sample, 13.5 % of them carry a suffix such as "- Remastered 2011" or "- Live", against 0.1 % of the hits, and genre words (*blues, metal, jazz*) and non-English words mark the niche catalogue. Two controls measure this. On the genre-matched sample (songs with both a preview and a Spotify genre, 20,610, from which as many hits as flops are drawn inside every genre family: 7,554 songs; Appendix J) the title still adds +2.9 pts to the final model; once suffixes, brackets and "feat." are stripped, +1.9 pts ($p = 0.002$, not significant after Bonferroni; Appendix J). The title reveals how the data were collected, not what makes a hit. A 185-dimensional *librosa* representation added +3.8 pts on one split ($p = 0.002$) but +0.9 pts (n.s.) under cross-validation. *FFT* descriptors of 30-second previews add +1.4 pts on the whole benchmark ($p = 4 \times 10^{-9}$), but nothing (+0.1 pts) when hits and non-hits come from the same genres: they recognise the excluded genres. The artist's *genre* itself would add +10.1 pts, but it is assigned by Spotify afterwards. *Release year* looks strong on ISMIR (+3.1 pts) only because hits there are older than the non-hit pool. *Market-trend* features (Sec. V-H), `chorus_hit`/`sections` and title length add nothing. Every test, table and figure of the audit is in Appendix J.

G. Lyrics on ISMIR 2019

ISMIR songs have lyrics in the official musiXmatch release (word counts over 5,000 words). We add two input blocks (9 numbers) to the final ISMIR model: eight lexical features (log number of words, log number of distinct words, richness, repetition, and the shares of *I/me/my*, *you/your*, *love* and common English words) and the probability given by a bag-of-words model trained inside each fold. Two shortcuts are removed first: only songs *with* lyrics are used on both sides (56 % of hits have lyrics, 36 % of non-hits), and the test is repeated on English songs only (97.5 % of hits, 80.5 % of non-hits). With year-matched negatives (protocol B, the one on which ISMIR inputs are judged, Sec. IV-F) lyrics add +5.1 pts ($p = 2 \times 10^{-5}$), and still +4.4 pts with English songs only and no artist shared between training and test ($p = 3 \times 10^{-5}$, 10/10 folds; Fig. 9a). With random negatives (protocol A) the gains are +4.7 pts ($p = 3 \times 10^{-4}$) and +2.7 to +3.0 pts in the English and unseen-artist variants ($p = 6 \times 10^{-4}$): consistent, but only indicative under Bonferroni. The words with the largest weights (Fig. 9b) are *love*, vocables (sung syllables without meaning: *la, oh, yeah*) and direct address on the hit side, and darker words (*kill, hate, crime*) on the other: lyrics partly identify mainstream themes and genres. Details: Appendix K.

H. Time series: chart dynamics

Treating the chart as a time series answers two questions. *Before release*, we built market-trend features (how close a song's audio is to the previous decade's hits and to the direction in which hits are moving): they add nothing significant to the final model (+0.07 pts, $p = 0.66$; the 13 per-descriptor deviations +0.46 pts, $p = 0.006$, indicative) because they mostly encode the era. The chart itself is a time series too: the number of weekly debuts is mostly trend, the number of holiday songs almost purely seasonal; their decomposition, periodogram and ARIMA forecasts are in Appendix L. *After release*, the question changes.

After release: early trajectories. A different question can be asked once a song has entered the chart: will it become a big hit? We take the 31,925 songs with a complete history (debut after the first chart and at least 52 weeks of follow-up; 16.4 % reach the top 10) and describe each by its first k weeks on the chart. Concretely, "the first k weeks" are the k chart positions the song holds in the k weekly charts that start with its debut: week 1 is the debut chart, week 2 the next week's chart, and so on, with position 101 if the song has dropped off. Madonna's *Vogue*, for example, debuted at 39, then held 28 and 12 (so for $k = 3$ the input is $(39, 28, 12)$) and later peaked at number 1. From these positions we also compute the best rank so far, the average climb per week (slope), the change of the weekly move (acceleration) and the number of weeks on the chart, and we add as context the year, month, and the artist's earlier songs and earlier top-10 hits reached before the debut (Random Forest, 200 trees, 5×3 CV). Table XIX shows that the trajectory dominates: three weeks plus context give an AUC of 0.892 (the three weeks alone 0.845) against 0.593 for the context alone (the weeks add +0.299 AUC, 15/15 folds, $p = 2 \times 10^{-16}$), and context adds +0.03 to +0.05 on top (15/15 folds, $p < 10^{-6}$ for every k). The prediction is not limited to songs already at the top: among the songs not yet in the top 10 after three weeks, the AUC is 0.870, and a model trained on songs that debuted before 2000 reaches 0.852 on songs from 2000 onwards ($k = 3$, with context, the same 200-tree forest fitted once). As for online content [14], early popularity predicts later popularity. This task differs from that of Sec. V-A: every song has already charted, so it separates big hits from small ones and cannot be used before release. A second target, at least 20 weeks on the chart, depends on chart rules that remove older songs and is not reported.

What happens in the first three weeks. Fig. 10 shows why so few weeks are enough. Songs that will reach

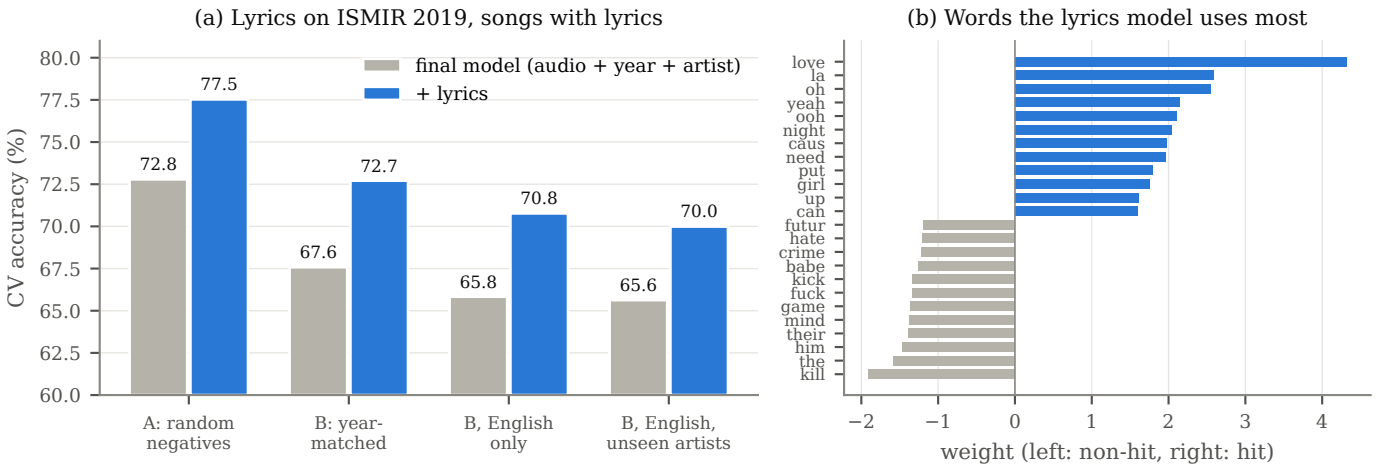


Fig. 9. Lyrics on ISMIR 2019. (a) Cross-validated accuracy of the final model without (gray) and with (blue) the lyrics inputs, from the plain protocol to the strictest control (year-matched, English only, no artist in both training and test folds). (b) Largest weights of the bag-of-words model fitted on the year-matched English sample (for display only, stemmed words used in at least 30 songs; the model itself uses all 5,000; "caus" = cause, "futur" = future).

the top 10 already differ at their debut (median position 73 against 87), but the gap opens in the following weeks: their median position climbs to 53 in week 2 and 39 in week 3, while the other songs stay around 80 and one in four has already left the chart by week 3 (Fig. 10a). The climb matters as much as the starting point (Fig. 10b): a song that debuts between 61 and 80 and loses ground reaches the top 10 3 % of the time, but 55 % of the time if it climbs 30 places or more by week 3. Even among debuts in the top 40, the share is 28 % for songs that fall and 82–100 % for songs that climb at least ten places. Permutation importance of the $k = 3$ model (Fig. 10c, held-out AUC 0.893) confirms this. Shuffling the week-3 position costs 0.088 AUC, the climb per week 0.029 and the best rank 0.025, whereas the debut position alone costs 0.004 once the later weeks are known. Among the context inputs, only the year matters (0.023), because chart dynamics changed over time; the artist's earlier songs cost less than 0.01. After release, then, it is the song's own momentum, not the artist's past, that tells a big hit from a small one.

VI. Discussion

The final model. After the audit, the model we retain is the main model of Sec. V-A: the frozen Random Forest (80 trees, 8 candidate features per split) on 20 inputs per song, namely the 13 Spotify descriptors, six decade indicators and artist history $h(s)$. It reaches 0.832 accuracy and 0.918 ROC-AUC on the Hit Predictor (Table III). On the external datasets the same recipe is used with each dataset's own audio descriptors, the release year or decade and artist history, with a 200-tree Random Forest (0.763 on ISMIR protocol A, 0.708 on protocol B, ROC-AUC 0.777 on SpotGenTrack). XGBoost is 1.3 points more accurate under cross-validation (0.845 vs 0.832, untested) but ties with the Random Forest on the canonical split ($p = 0.87$), so we keep the configuration that matches prior work. Every other candidate is left out for a stated reason (Table V). A separate model answers a different question, for songs already on the chart: the same kind of Random Forest on the first weeks of the chart trajectory (Sec. V-H). Where lyrics are available (ISMIR, songs with musiXmatch lyrics), adding them gives the best model we found on those songs (0.775 in protocol A, 0.727 in protocol B, against 0.728 and 0.676 for the final model on the same songs; Table XVIII).

How much of the gain is real? The benchmark's negative class excludes the artists of that decade's hits. Our feature only counts *earlier* decades, so it is leakage-free in time, but an artist who charted before is likely to chart again and is therefore unlikely to appear as a negative by construction: flops have on average 0.037 earlier hits, against 3.21 for hits. The +5.0 pts on this benchmark, and the head-to-head gains of Table XII, should be read as benchmark-specific. The ISMIR year-matched protocol, whose negatives are chosen neither by artist nor by date, gives a more conservative +2.6 pts; this is the number we consider transferable.

Era as a shortcut. On ISMIR, release year looks like a strong predictor (70.2 % alone, and +12 to +13 points in the original study) because hits are older than the non-hit pool. Once years are matched, it carries no information. On the Hit Predictor, the decade is uninformative alone and helps only through interactions, which is consistent with era-specific tastes. It could also reflect decade-dependent genre rules of the negative class, but on the genre-matched sample the decade still adds +2.5 pts to the 13 descriptors (25/25, $p = 8 \times 10^{-7}$; Table XVI), so genre is not the whole story. Reported gains from date features should be accompanied by a year-matched control.

Genre as a shortcut. On the Hit Predictor, adding

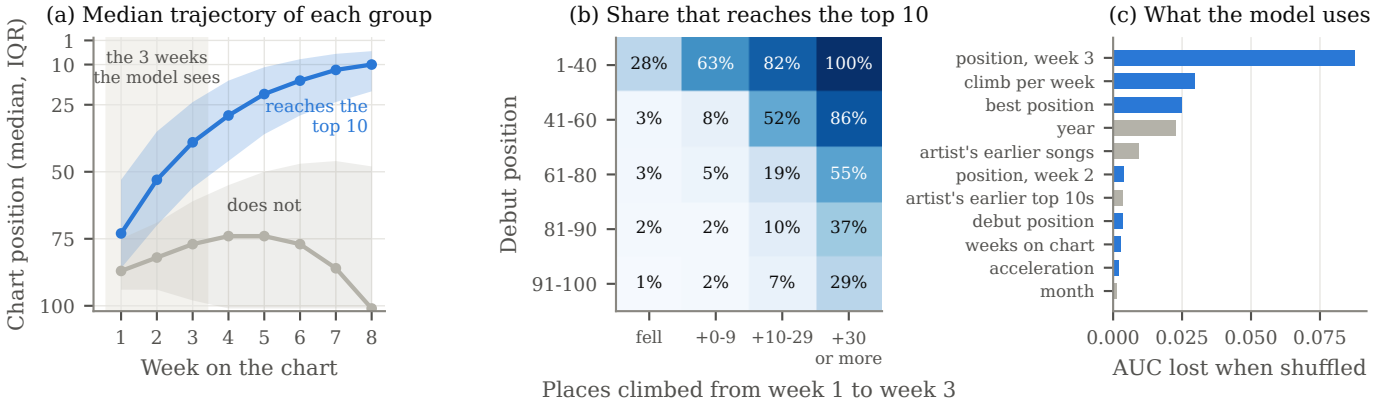


Fig. 10. The first weeks on the Hot 100 (31,925 songs, 5,238 reach the top 10). (a) Median chart position (line) and interquartile range (band: the middle 50 % of songs) in each week after the debut; 101 = off the chart. The shaded weeks are the inputs of the $k = 3$ model. (b) Share of songs that eventually reach the top 10, by debut position and by places gained between week 1 and week 3 ("fell": lost places or left the chart). (c) Permutation importance of the inputs of the $k = 3$ model on a held-out 25 %: drop in ROC-AUC when one input is shuffled. Blue: trajectory; gray: context.

TABLE V

INPUTS OF THE FINAL MODEL, AND THE CANDIDATES LEFT OUT. GAINS IN ACCURACY POINTS (5×5 CV ON THE HIT PREDICTOR; ROWS MARKED ISMIR: 5×2 CV ON ISMIR). BASELINES: KEPT INPUTS, OVER THE 13 DESCRIPTORS (40,560 SONGS); TITLE, TITLE LENGTH AND `CHORUS_HIT`, OVER THE 13 DESCRIPTORS (THE TITLE'S "AFTER CONTROLS" VALUE: OVER THE FINAL MODEL ON THE GENRE-MATCHED SAMPLE, TITLES CLEANED); RICH LIBROSA, OVER THE 13 DESCRIPTORS ON THE 3,562-SONG AUDIO SUBSET; FFT, GENRE AND MARKET TREND, OVER THE FINAL MODEL (ON THE SONGS WITH A PREVIEW, RESP. A GENRE).

Input	Evidence	Decision
13 Spotify descriptors	0.782	kept (baseline)
Decade (6 indicators)	+2.0	kept; helps new artists
Artist history $h(s)$	+2.9; ISMIR B +2.6	kept; survives every control run on it (each separately)
Song title (TF-IDF)	+6.7; +1.9 after controls ($p = 0.002$, not Bonferroni-significant)	out: construction artefact
Title length	+0.2, n.s.	out: no signal
chorus_hit, sections	0.0, n.s.	out: no signal
Rich librosa (PCA 40)	+0.9, n.s.	out: single-split luck
FFT (31)	+0.1 genre-matched	out: genre shortcut
Spotify artist genre	+10.1	control only: assigned afterwards
Market trend	+0.07, n.s.	out: carries only the era
Lyrics (bag of words + 8 lexical)	ISMIR B +4.4, unseen artists	kept where lyrics exist (ISMIR); unavailable for the Hit Predictor

the artist's genre takes the main model to 0.928, and the apparent FFT gain disappears once genre is controlled (Appendix J). Any audio representation evaluated on this benchmark can score by recognising the excluded genres rather than by modelling what makes a song a hit, so audio gains on it should be reported together with a genre-matched control. Artist history passes that control: on the genre-matched sample, decade and artist history still add +6.7 pts to the 13 descriptors (25/25, $p = 9{\times}10^{-13}$; Table XVI). FFT does not. Within the audio content itself, therefore, no representation we tested adds to artist history and era once the construction shortcuts are controlled. The one content input that does is the lyrics (Sec. V-G), whose gain survives year matching, an English-only control and unseen artists; their strongest words, however, suggest that they too partly identify mainstream genres and themes, and a genre-matched control for lyrics remains to be done. On these data, the 13 audio descriptors carry most of the accuracy, but what adds to them and survives the controls is who sings the song, when, and what it is about—not a richer description of how it sounds.

Single splits and multiple comparisons. Several of our own earlier single-split conclusions did not survive cross-validation, and with a few hundred p-values some small ones are expected by chance; this is why the survival ranking requires the Bonferroni threshold. Details: Appendix M.

Survival ranking. Table VI lists every gain we tested, split into three groups. The groups are assigned in this order. *Did not survive*: p above the Bonferroni threshold, whatever the control (a failed control is then noted in the verdict). *Shortcut* (leaks included): significant after Bonferroni, but failing a control (under at least one control its gain is no longer Bonferroni-significant) or not known on release day, so the gain is real on that dataset but would not transfer. *Survived*: significant after Bonferroni and failing none of the controls run on it (a ✓ where a control was run). Only artist history, era, the lyrics and, after release, the early trajectory are in the survived group. The table groups by outcome; Fig. 5 instead lists each rejected

TABLE VI

SURVIVAL RANKING: EVERY TESTED GAIN, RANKED BY HOW FAR IT SURVIVES. *BONF.*: SIGNIFICANT AFTER THE BONFERRONI CORRECTION OVER AT MOST 400 TESTS ($p < 0.05/400 = 1.25 \times 10^{-4}$, SEC. IV-E). *CONTROL*: ✓ PASSED THE CONTROL THAT REMOVES THE KNOWN SHORTCUT OF ITS DATASET (YEAR-MATCHED OR GENRE-MATCHED NEGATIVES, UNSEEN ARTISTS); × FAILED IT, OR THE INPUT IS NOT KNOWN ON RELEASE DAY; – NO SUCH CONTROL APPLIES OR NONE WAS RUN (THE EARLY-TRAJECTORY ROW ANSWERS A SEPARATE, AFTER-RELEASE QUESTION AND IS EXEMPT FROM THE RELEASE-DAY RULE). Δ: ACCURACY POINTS UNLESS STATED; p: CORRECTED t-TEST (MCNEMAR FOR SINGLE SPLITS). NOT RANKED: THE FUTURE-DECADES CHECK OF ARTIST HISTORY (SINGLE SPLIT PER DECADE, MCNEMAR, +1.9 TO +3.4 PTS, $p \leq 10^{-7}$), A ROBUSTNESS CHECK OF THE FIRST ROW.

	Gain tested	Δ	p	Bonf.	Control	Verdict
survived	Decade + artist history, Hit Predictor	+5.0	$7{\cdot}10^{-17}$	✓	✓	real; benchmark-inflated, since these controls leave the artist rule of the negatives in place (ISMIR B is the control for it); unseen artists +5.0 (10^{-13}), genre-matched +6.7 ($9{\cdot}10^{-13}$)
	Every classifier, decade + artist (5 models)	+4.6 to +5.7	$\leq 4{\cdot}10^{-5}$	✓	–	gain comes from the inputs, not the model
	Head-to-head, Du's protocol + artist (his RF and LR, our frozen RF; 2 decades)	+2.1 to +3.2	$\leq 6{\cdot}10^{-6}$	✓	–	better on 50/50 splits; benchmark-specific
	Decade, Hit Predictor	+2.0	10^{-10}	✓	✓	genre-matched +2.5 ($8{\cdot}10^{-7}$); unseen artists +2.0 (10^{-8}); helps new artists (+2.2)
	Artist history, Hit Predictor	+2.9	10^{-11}	✓	✓	unseen artists +3.0 ($2{\cdot}10^{-9}$); genre-matched +4.2 ($2{\cdot}10^{-8}$)
	Artist history, ISMIR A (vs audio)	+2.5	$2{\cdot}10^{-5}$	✓	✓	real (year-matched: next row)
	Artist history, ISMIR B year-matched	+2.6	$3{\cdot}10^{-5}$	✓	✓	**transferable estimate**; unseen artists +2.9 ($9{\cdot}10^{-7}$)
	Lyrics, ISMIR B, English, unseen artists	+4.4	$3{\cdot}10^{-5}$	✓	✓	only content input that survives
	Early trajectory: 3 weeks added to context (AUC)	+.299	$2{\cdot}10^{-16}$	✓	–	after release only; AUC .593 → .892 (15/15 folds); context on top of the weeks: +.047 ($< 10^{-6}$)
shortcut	Song title (TF-IDF), Hit Predictor	+6.7	$4{\cdot}10^{-20}$	✓	×	formatting + genre words; genre-matched and cleaned: +1.9 ($p = .002$)
	Spotify artist genre, Hit Predictor	+10.1	$8{\cdot}10^{-23}$	✓	×	not known on release day; control only
	FFT, whole Hit Predictor	+1.4	$4{\cdot}10^{-9}$	✓	×	genre-matched +0.1 ($p = 0.84$); given genre −1.1 ($5{\cdot}10^{-7}$)
	Decade, ISMIR A	+2.9	$1.1{\cdot}10^{-4}$	✓	×	year-matched: −0.2 ($p = 0.56$)
did not survive	Release year, ISMIR A	+3.1	$2{\cdot}10^{-4}$	×	×	also a shortcut: year-matched −0.1 ($p = 0.66$)
	Decade + artist, SpotGenTrack (AUC .690→.777)	+1.3	$3{\cdot}10^{-4}$	×	×	also a leak: popularity measured today; upper bound
	Lyrics, ISMIR A, English, unseen artists	+3.0	$6{\cdot}10^{-4}$	×	–	indicative; the year-matched test (B) is stronger
	Lyrics lexical, SpotGenTrack (5×1)	+0.6	.002	×	–	indicative; confirmed on ISMIR
	Market trend, 13 per-feature deviations	+0.46	.006	×	–	indicative only
	ARIMA vs seasonal naive (11 origins; MASE = mean absolute error divided by that of the seasonal naive forecast on the training months, Appendix O)	MASE −0.5	.006	×	–	indicative only
	FFT on top of artist, audio subset	+2.0	.008	×	–	genre shortcut (FFT row above)
	Decade, audio subset	+1.0	.03	×	–	small sample
	Audio + artist + FFT vs final, whole benchmark	+0.4	.04	×	×	no
	Rich librosa (PCA 40), audio subset	+0.9	.15	×	–	single split said +3.8 ($p = .002$)
	Artist history for new artists, Hit Predictor	+0.4	.17	×	–	does not help: $h(s) = 0$ for all of them
	Title length	+0.2	.28	×	–	no signal
	Market trend, distance + direction	+0.07	.66	×	–	carries only the era
	`chorus_hit` + `sections`	0.0	.82	×	–	no signal

input at the first question it fails, so an input that fails both a control and Bonferroni (release year on ISMIR, the SpotGenTrack leak) appears in the figure under "shortcut" or "leak" and in the table under "did not survive" with the failed control noted.

Limitations. Artist history is matched by normalised name, not by a canonical identifier, so homonymous artists can be merged. On SpotGenTrack the "hits" are songs with Spotify popularity ≥ 55, a score measured when the dataset was collected, not at release; artist history there counts the artist's earlier songs that are popular *today*, so it is not strictly known on release day, and the SpotGenTrack gain should be read as an upper bound. The Hit Predictor and ISMIR results do not have this problem. Artist history does not help new artists on the Hit Predictor and at most modestly on ISMIR (+1.4 pts, indicative), and it partly measures cumulative advantage [17] rather than the intrinsic appeal of a song. We use classical classifiers on fixed descriptors rather than deep networks on raw audio [3], [6]: our question is which inputs carry signal under a controlled protocol, and the comparison with prior work is made at equal model; a learned representation could still capture what the descriptors miss. The comparison with Zangerle et al. uses a classifier, whereas they trained a regression on peak chart position (non-hits set to 150) and report as accuracy the share of songs placed on the right side of position 100; we also use a different random draw of negatives; the comparison with Du uses re-drawn splits. On ISMIR the period of a song is its release year in the Million Song Dataset, which for a reissue or a compilation can be later than its chart run, so "earlier" hits there are only as reliable as that year. On the Hit Predictor the decade of a hit is the decade in which it charted, which for the few songs whose run crosses a decade boundary is later information than the release decade. Our audio previews are 30-second clips from a different service than

the one that computed the Spotify descriptors.

VII. Future Work

Appendix J already runs the cheap versions of several follow-ups: artist matching (no gain available), a genre control for audio features, model ageing across decades and lexical lyrics descriptors. What remains needs new data. (i) *Cold start*: artist history barely helps first-time artists on the benchmark, and the FFT gain for them was a genre shortcut; features such as label, collaborators, prior streams or social media remain to be tested, with a genre-matched control. (ii) *Lyrics and emotion*: lyrics for the Hit Predictor songs (matched by title and artist), a genre-matched control for the lyrics gain, semantic and affective lyrics representations, and learned audio embeddings, evaluated with the same audit protocol. (iii) *Benchmark design*: a hit-prediction dataset whose negatives are sampled independently of artist, genre and date. Our genre-matched sample is a small, post-hoc approximation of it; a purpose-built one would let content features be compared without construction shortcuts. (iv) *Early trajectories*: Sec. V-H uses weekly chart positions only; daily streams and radio plays would allow the same prediction days after release.

VIII. Conclusion

A song's artist history is a simple, leakage-free signal that improves hit prediction across classifiers, beats the only prior model on the Spotify Hit Predictor benchmark in a head-to-head comparison, and transfers to an independent peer-reviewed dataset where it exceeds the best published accuracy on the original protocol and survives a year-matched control. These gains hold for artists the model never saw in training, which rules out artist recognition; but artist history does not help an artist with no earlier hit on the benchmark, where for those newcomers only era context helps (on ISMIR there is an indicative hint, $p = 0.002$, that an empty history is informative); spectral descriptors of a 30-second preview appear to help them too, but that gain disappears under a genre control. The same audit shows that larger gains—song titles, release dates, genre-driven audio features, rich audio on a single split—can come from how the data were built or from chance. Chart time series add nothing before release, but after release a song's first weeks are the strongest predictor we found. The lyrics are the only content input whose gain survives every control we could run on them (year matching, English only, unseen artists). Knowing which gains are real matters more than the headline number.

Code and data availability

Code. https://github.com/uzznat/hit-song-prediction. Every number in this paper is produced by the scripts in `repro/` (shared data loading and evaluation in `common.py`; one script per section, Table XX) and written to `paper_results/`. Random seeds are fixed (42 unless stated). `docs/DATA.md` explains where each dataset goes.

Data. All datasets are public: Spotify Hit Predictor [16], https://github.com/fortyTwo102/The-Spotify-Hit-Predictor-Dataset (also Kaggle: https://www.kaggle.com/datasets/theoverman/the-spotify-hit-predictor-dataset); Billboard Hot 100 weekly charts, https://github.com/utdata/rwd-billboard-data (file `hot-100-current.csv`, downloaded in July 2026; charts of 4 August 1958 to 11 July 2026); ISMIR 2019 hit-song data [4], https://zenodo.org/records/3258042; SpotGenTrack [5], https://data.mendeley.com/datasets/4m2x4zngny; musiXmatch lyrics for the Million Song Dataset, http://millionsongdataset.com/musixmatch/. Audio previews were retrieved from the public Deezer API (https://developers.deezer.com/api). The previews and the features extracted from them are not in the repository (the audio is copyrighted); `repro/fetch_audio_full.py` downloads and processes them again, but Deezer's search results can change over time, so a re-run may find a slightly different set of songs.

References

[1] F. Pachet and P. Roy, "Hit song science is not yet a science," in *Proc. 9th International Conference on Music Information Retrieval (ISMIR)*, 2008, pp. 355–360.

[2] Y. Ni, R. Santos-Rodríguez, M. McVicar, and T. De Bie, "Hit song science once again a science?" in *4th International Workshop on Machine Learning and Music (MML), held with NIPS*, Sierra Nevada, Spain, 2011.

[3] L.-C. Yang, S.-Y. Chou, J.-Y. Liu, Y.-H. Yang, and Y.-A. Chen, "Revisiting the problem of audio-based hit song prediction using convolutional neural networks," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 621–625.

[4] E. Zangerle, R. Huber, M. Vötter, and Y.-H. Yang, "Hit song prediction: Leveraging low- and high-level audio features," in *Proc. 20th International Society for Music Information Retrieval Conference (ISMIR)*, Delft, The Netherlands, 2019, pp. 319–326.

[5] D. Martín-Gutiérrez, G. Hernández Peñaloza, A. Belmonte-Hernández, and F. Álvarez García, "A multimodal end-to-end deep learning architecture for music popularity prediction," *IEEE Access*, vol. 8, pp. 39 361–39 374, 2020.

[6] M. Nofer, V. Nimani, and O. Hinz, "DeepHits: A multimodal CNN approach to hit song prediction," *Machine Learning and Knowledge Extraction*, vol. 8, no. 3, p. 58, 2026.

[7] K. Middlebrook and K. Sheik, "Song hit prediction: Predicting Billboard hits using Spotify data," *arXiv preprint arXiv:1908.08609*, 2019.

[8] I. Dimolitsas, S. Kantarelis, and A. Fouka, "SpotHitPy: A study for ML-based song hit prediction using Spotify," *arXiv preprint arXiv:2301.07978*, 2023.

[9] M. Zhao, M. Harvey, D. Cameron, F. Hopfgartner, and V. J. Gillet, "An analysis of classification approaches for hit song prediction using engineered metadata features with lyrics and audio features," in *Information for a Better World (iConference 2023)*, ser. Lecture Notes in Computer Science. Springer, 2023, pp. 303–311.

[10] J. L. Du, "Spotify between decades: Hit or miss predictor," MSc thesis, Data Science & Society, Tilburg University, 2022, http://arno.uvt.nl/show.cgi?fid=160898.

[11] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[12] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[13] M. Interiano, K. Kazemi, L. Wang, J. Yang, Z. Yu, and N. L. Komarova, "Musical trends and predictability of success in contemporary songs in and out of the top charts," *Royal Society Open Science*, vol. 5, no. 5, p. 171274, 2018.

[14] G. Szabo and B. A. Huberman, "Predicting the popularity of online content," *Communications of the ACM*, vol. 53, no. 8, pp. 80–88, 2010.

[15] D. Herremans, D. Martens, and K. Sörensen, "Dance hit song prediction," *Journal of New Music Research*, vol. 43, no. 3, pp. 291–302, 2014.

[16] fortyTwo102, "The Spotify hit predictor dataset (1960–2019)," GitHub repository, https://github.com/fortyTwo102/The-Spotify-Hit-Predictor-Dataset, accessed Sep. 2026.

[17] M. J. Salganik, P. S. Dodds, and D. J. Watts, "Experimental study of inequality and unpredictability in an artificial cultural market," *Science*, vol. 311, no. 5762, pp. 854–856, 2006.

[18] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[19] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[20] B. L. Sturm, "A simple method to determine if a music information retrieval system is a "horse"," *IEEE Transactions on Multimedia*, vol. 16, no. 6, pp. 1636–1644, 2014.

[21] A. Flexer, "A closer look on artist filters for musical genre classification," in *Proc. 8th International Conference on Music Information Retrieval (ISMIR)*, 2007, pp. 341–344.

[22] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[23] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[24] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 18–24.

[25] D. Bogdanov, N. Wack, E. Gómez, S. Gulati, P. Herrera, O. Mayor, G. Roma, J. Salamon, J. R. Zapata, and X. Serra, "Essentia: An audio analysis library for music information retrieval," in *Proc. 14th International Society for Music Information Retrieval Conference (ISMIR)*, 2013, pp. 493–498.

[26] T. Bertin-Mahieux, D. P. W. Ellis, B. Whitman, and P. Lamere, "The million song dataset," in *Proc. 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011, pp. 591–596.

[27] G. Rompolas, A. Smpoukis, E. Kafeza, and C. Makris, "Predicting song popularity through machine learning and sentiment analysis on social networks," in *Artificial Intelligence Applications and Innovations. AIAI 2024 IFIP WG 12.5 International Workshops*, ser. IFIP Advances in Information and Communication Technology. Springer, 2024, pp. 314–324.

[28] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.

[29] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[30] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.

Appendix: technical details

The paper ends with the references above. The appendices below are not needed to follow it: they give every detail needed to rebuild each result (inputs, settings, extra tables, controls, statistics, code) and a glossary.

A Inputs in detail
B Models and settings
C Extended related work
D Datasets in detail
E Main result: more detail
F PCA, everything together
G Head-to-head with Du
H External validation
I Unseen artists, cold start
J Audit and further experiments
K Lyrics on ISMIR
L Chart components, trajectories
M Single splits, multiple comparisons
N Which script produces what
O Time-series details
P Glossary
Q Questions a reader may ask

Appendix A
Inputs in detail

TABLE VII
The 13 Spotify audio descriptors (Spotify Web API definitions) and how well each one alone separates hits on the Hit Predictor (ROC-AUC of the raw value, reported as $\max(\text{AUC}, 1-\text{AUC})$ so that a descriptor lower in hits also scores above 0.5; Sec. V-B).

Descriptor	Meaning	AUC
instrumentalness	likelihood that the track has no vocals (0–1); lower in hits	.74
danceability	suitability for dancing from tempo, rhythm stability, beat strength (0–1)	.70
loudness	average loudness in dB	.64
valence	musical positiveness, sad (0) to happy (1)	.64
acousticness	confidence that the track is acoustic (0–1); lower in hits	.60
energy	perceived intensity and activity (0–1)	.59
speechiness	presence of spoken words (0–1)	.58
mode	major (1) or minor (0)	.54
time signature	estimated beats per bar (integer, 0–5 in these data)	.53
liveness	probability that the track was recorded live (0–1)	.53
tempo	estimated beats per minute (about 0–240)	.52
duration	length in milliseconds	.51
key	pitch class of the track (0 = C, ..., 11 = B)	.51

Decade. Six indicators (1960s ... 2010s), exactly one equal to 1. On ISMIR, the release year as a number; on SpotGenTrack, the release decade as a number (1990 for 1990–1999).

Artist history. Defined in Sec. IV-B and Fig. 4. On the Hit Predictor and the chart, artist names are processed in this order: accents removed (Unicode NFKD decomposition, then non-ASCII characters dropped) and lowercased; cut at the first separator, which must be written with a space on each side, i.e. a whole word (so even the comma counts only as " , "; "Earth, Wind & Fire" is cut at " & " into "earth wind"), among ("featuring", "feat.", "ft.", "with", "&", "x", "+", "/", "vs.", "and" or a comma; so "Brandy" is not cut); every character other than a–z, 0–9 and space removed, and spaces collapsed. So "Madonna feat. Justin Timberlake" counts for Madonna. On the Hit Predictor the reference set is the weekly Hot 100 since August 1958 (so the 1958–59 charts form a "1950s" decade that counts as earlier for 1960s songs): for every artist and every decade (the decade of the chart week), we collect the set of distinct lower-cased titles, and $h(s)$ sums the sizes of these sets over the decades strictly before the song's decade. On ISMIR and SpotGenTrack it is the set of positive songs of the dataset, counted over strictly earlier release years. ln is the natural logarithm.

Rejected candidates. *Song title*: TF-IDF of word uni- and bigrams, logistic regression, out-of-fold probability as one feature. *Title length*: number of words. *chorus_hit, sections*: two fields of the Hit Predictor computed from Spotify's audio-analysis endpoint (inherited from the Echo Nest, the music-analysis company Spotify bought), the estimated start time of the chorus (the author's estimate: the start of the third section) and the number of sections (verse, chorus, ...) of the track.

Audio from the 30-second previews. Every preview is decoded to mono at 22,050 Hz (first 30 s). All frame-based descriptors are computed with librosa 0.11 [24], an open-source Python library for audio analysis, with its defaults: a short-time Fourier transform with a 2,048-sample window (93 ms) and a hop of 512 samples (23 ms). *FFT* (31; the name only says that every descriptor is derived from the fast Fourier transform of the clip, i.e. its spectrogram): mean and standard deviation over frames of the spectral centroid, bandwidth, roll-off (85 %), flatness and contrast (contrast averaged over its 7 frequency bands, so one value per frame); the mean of each of 13 MFCCs; the mean and standard deviation of the 12-bin STFT chroma, taken over all bins and frames together (two numbers); and six rhythm descriptors: tempo estimated from the onset envelope, mean and standard deviation of the onset envelope, *pulse clarity* (the maximum divided by the mean of the time-averaged tempogram, i.e. how strongly one periodicity dominates), and the mean and coefficient of variation (s.d./mean) of inter-onset intervals. *Rich librosa* (185): the mean and standard deviation over frames of each row of 20 MFCCs and of their first and second derivatives (120), 12 constant-Q chroma bins (24), 6 tonnetz dimensions of the harmonic part (12), 7 spectral-contrast bands (14), spectral centroid, bandwidth and roll-off (6), flatness (2), zero-crossing rate (2) and RMS

energy (2), plus the onset-envelope mean and standard deviation and the pulse clarity (3); reduced to 40 principal components inside each training fold.

Genre (16 families, control only). Spotify artist tags (from the SpotGenTrack artist file, matched by artist name normalised as on the Hit Predictor; when a name occurs twice, the artist with more followers is kept) are mapped by substring to families: hip-hop (hip hop, rap, trap, drill, grime, crunk, bounce), R&B/soul (r&b, soul, funk, motown, disco, new jack, quiet storm, doo-wop), country (country, bluegrass, americana, honky, western swing), rock (rock, punk, grunge, emo, britpop, new wave, shoegaze, post-), metal (metal, hardcore, deathcore, grindcore, thrash, doom), electronic (edm, house, techno, trance, electro, dubstep, drum and bass, dance, idm, ambient, downtempo, breakbeat, glitch, synth), pop (pop, boy band, girl group, teen, adult standards, easy listening), jazz (jazz, bebop, swing, big band, bop, exotica, lounge), classical (classical, orchestra, baroque, opera, romantic era, early music, choral, compositional, minimalism, soundtrack, score), folk (folk, singer-songwriter, acoustic, cantautor), blues, Latin (latin, reggaeton, salsa, bachata, cumbia, mexican, tango, bossa, samba, mpb, sertanejo, flamenco, ranchera, banda, tropical), reggae (reggae, dancehall, ska, dub, rocksteady), gospel (gospel, christian, worship, ccm, spiritual), world (afro, k-pop, j-pop, indian, bollywood, arab, turkish, greek, celtic, world, chanson, schlager, french, german, italian, brazil, polka, nordic, russian, african, hawaiian), and other (no match). A tag is assigned to every family one of whose keywords it contains (so "rocksteady" counts for rock and reggae, "k-pop" for pop and world). An artist gets one yes/no input per family; the *dominant* family is the one matched by the most tags (ties go to the family listed first above), and it is used for the genre-matched sample, in which, for every dominant family ("other" included), we draw at random (seed 42) as many hits as flops (the smaller of the two counts).

Market trend. Descriptors are standardised over the whole dataset. For each decade we compute the centroid (mean vector) of its hits. A song's *distance* is the Euclidean distance between its descriptors and the centroid of the previous decade's hits; its *direction* is the projection of that difference on the unit vector pointing from the centroid two decades back to the one of the previous decade (dot product divided by the vector's length); the 13 *deviations* are the coordinates of the difference. The 1960s have no previous decade (distance and deviations set to the median and to 0), and the direction exists from the 1980s on (median before). These features are computed once on the whole dataset, from earlier decades' hits only, not inside the folds; since they add nothing, this does not affect any conclusion.

Lyrics (ISMIR). From musiXmatch word counts, which cover only the 5,000 most frequent stemmed words (so all counts below are over that vocabulary, not the full text): ln(1 + number of words), ln(1 + number of distinct words), richness (distinct/total), repetition (count of the most frequent word/total), shares of *I, me, my*, of *you, your*, of *love*, and of 29 common English words, matched as written against the stemmed vocabulary (28 of them occur in it; *don't* does not and is ignored) (*the, i, you, and, to, a, me, my, it, in, of, is, that, your, on, be, we, all, love, do, so, no, for, what, with, can, this, but, don't*); plus the out-of-fold TF-IDF logistic-regression probability. A song counts as English if at least 25 % of its words are in that list.

Early trajectory. A "song" is one title–performer pair of the weekly chart file. Week w is the chart published $w-1$ weeks after the debut chart; p_w is the song's position in it (101 = not on the chart). For k weeks: the k positions, the best of these k positions, the number of these k weeks in which the song is on the chart (nothing after week k is used), the slope $(p_k - p_1)/(k-1)$ (negative = climbing; only for $k \geq 2$) and the acceleration $(p_k - p_{k-1}) - (p_{k-1} - p_{k-2})$ (only for $k \geq 3$); context: debut year and month, ln(1+earlier songs of the artist, i.e. songs that debuted before this one) and ln(1+earlier songs of the artist that had already reached the top 10 before the debut), the artist being the performer field normalised as on the Hit Predictor. The "not yet top 10" column of Table XIX comes from a model retrained, with the same settings, on those songs only. Permutation importance (Fig. 10c) uses one stratified 75/25 split (seed 42). The target is whether the song's peak over its whole chart run is in the top 10; songs are kept only if they debuted after the first chart (August 1958) and at least 52 weeks before the last one, so that their run is almost always complete (a song that peaks for the first time more than a year after its debut is rare).

Appendix B
Models and settings

TABLE VIII
EVERY MODEL USED, WITH ALL NON-DEFAULT SETTINGS. NONE WAS TUNED ON THE TEST DATA.

Model	Settings
Random Forest (main)	80 trees, $\min(8, d)$ candidate features per split for a model with d inputs (8 for the final model; all of them when a block has fewer than 8), Gini, fully grown trees, seed 42 (configuration of [7])
Random Forest (external, trajectories)	200 trees, $\sqrt{d}$ candidate features per split
XGBoost	400 trees, depth 6, learning rate 0.05, subsample 0.8, column sample 0.8
Logistic regression	standardised inputs, L2, $C = 1$
k-NN	standardised inputs, $k = 25$, Euclidean distance, equal votes
SVM	standardised inputs, RBF kernel $e^{-\gamma\|x-x'\|^2}$, $C = 1$, $\gamma = 1/(d \cdot \mathrm{Var}(X))$ (scikit-learn "scale")
Du's models [10]	RF 450 trees, $m_{\mathrm{try}} = 2$, at most 30 leaves; logistic regression on standardised inputs (L2, $C = 1$); one model per decade
Title text model	TF-IDF of word uni- and bigrams (words in at least 2 titles, sublinear term frequency $1 + \ln \mathrm{tf}$) + logistic regression $C = 4$; inner 5-fold out-of-fold
Lyrics text model	TF-IDF (sublinear, no minimum document frequency) on the musiXmatch counts of all 5,000 stemmed words + logistic regression $C = 1$; inner 5-fold out-of-fold
Rich audio	standardisation + PCA (40) fitted inside each training fold
Cross-validation	repeated stratified k-fold: 5×5 (Hit Predictor), 5×2 (ISMIR), 5×1 (SpotGenTrack), 5×3 (trajectories); stratified group k-fold by artist for the unseen-artist tests
Tests	corrected resampled t-test [19] for CV, two-sided (formula in Sec. IV-E); McNemar [18] for single splits; Bonferroni over at most 400 tests ($p < 1.25 \times 10^{-4}$)
Artist-grouped CV	scikit-learn `StratifiedGroupKFold` (5 folds, shuffled), run once per repeat with seed $42 + r$ for repeat $r = 0, 1, \ldots$, so 25 grouped folds on the Hit Predictor and 10 on ISMIR; folds then differ slightly in size, and the corrected t-test uses the mean $n_{\mathrm{test}}/n_{\mathrm{train}}$ over folds
Audio-subset single split	stratified 75/25 (2,671 training, 891 test songs), seed 42
Seeds	42 for every split, sample and model; in repeated CV, one seed (42) generates the different shuffles of all the repeats
Software	Python 3.14, scikit-learn 1.8, XGBoost 3.2, librosa 0.11, pandas 3.0, NumPy 2.4, SciPy 1.17, statsmodels 0.15

APPENDIX C
EXTENDED RELATED WORK

Audio-based hit prediction. Pachet and Roy [1] found that acoustic and human-annotated descriptors could not predict popularity, while Ni et al. [2] and Herremans et al. [15] reported above-chance results on chart data. Yang et al. [3] revisited the problem with convolutional networks on audio. Zangerle et al. [4] combined low- and high-level Essentia descriptors [25] of Million Song Dataset tracks [26] in a wide-and-deep network and reached 75.04 % accuracy on a balanced set of 11,664 songs (5,832 hits; one is lost to a missing year or incomplete descriptors, so we keep 5,831, Appendix D), noting that adding the release year improved every configuration.

Artist reputation. Interiano et al. [13] analyse more than 500,000 UK songs (1985–2015) and show that a superstar indicator improves the prediction of chart success over acoustic features alone. Salganik et al. [17] showed experimentally that success in cultural markets depends on social influence and cumulative advantage, which is one reason why an artist's past success predicts future success.

Spotify descriptors and Billboard labels. Middlebrook and Sheik [7] matched about 24,000 Spotify tracks (1985–2018) to the Hot 100 and reported 87.7 % test accuracy with a Random Forest; their feature list includes chart-derived fields and an artist past-performance feature. SpotHitPy [8] reported about 86 % on roughly 18,500 songs. Zhao et al. [9] added engineered metadata and lyrics features for Top-10 prediction (89.1 % with a Random Forest). These numbers come from different datasets and label definitions. The Spotify Hit Predictor dataset [16] is a public benchmark of decade-balanced hits and non-hits (1960–2019); the only academic study we found on it is an MSc thesis by Du [10], which reports 84 % (2000s) and 82 % (2010s) with one Random Forest per decade.

Multimodal and popularity prediction. Martín-Gutiérrez et al. [5] released SpotGenTrack and predicted Spotify popularity with an end-to-end multimodal network; DeepHits [6] combines log-Mel CNNs, lyric embeddings, artist popularity and release year on the same data; Rompolas et al. [27] use social-media sentiment.

Evaluation pitfalls. Dietterich [18] recommends McNemar's test for a single split, and Nadeau and Bengio [19] show that naive t-tests over overlapping resamples are anti-conservative and propose a corrected variance. Leakage [11] and shortcut learning [12] describe models that exploit cues unrelated to the intended task. In music information retrieval, Sturm [20] showed that systems can reach high scores through confounds in the ground truth ("horses"), and Flexer [21] showed that letting songs by the same artist appear in training and test sets inflates genre-classification accuracy; our artist-grouped cross-validation applies this artist filter to hit prediction. To our knowledge, no hit-prediction study on Spotify descriptors reports both a significance-tested head-to-head

on the same data and an audit of construction artefacts.

Appendix D
Datasets in detail

Spotify Hit Predictor. Six decade files (1960s–2010s) list 13 Spotify audio descriptors (danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, duration, time signature), two analysis fields (`chorus_hit`, `sections`), the artist, the title and a binary target [16]. The six files hold 41,106 rows; 537 track URIs (Appendix P) appear more than once, 450 of them in two decade files and 26 with different labels. We keep the first copy in the order the files are read (00s, 10s, 60s, 70s, 80s, 90s, i.e. alphabetical), which leaves 40,560 songs; a song's decade is the decade of the file that copy comes from. A song is a hit if it appeared on the weekly Hot 100 in that decade. Crucially, the dataset documentation states that a non-hit must satisfy all of: the track is not in that decade's hit list, *its artist is not in that decade's hit list*, it belongs to a "non-mainstream and/or avant-garde" genre, and that genre has no song in the hit list. Each decade file is therefore balanced (48.3–49.9 % hits), and the negative class is selected partly *by artist* and *by genre*. We return to this in Sec. VI.

Audio subset. We first drew a random sample of 3,996 songs, the same number (333) in each of the 12 decade × hit/non-hit cells (seed 42), and searched each one on the public Deezer API with the query `artist:"<artist>" track:"<title>"` (the raw artist and title fields of the dataset, with double quotes and backslashes removed), keeping the 30-second preview of the first result (the quoted fields make Deezer require both the artist and the title; the match is not checked further, so a few previews may be another version of the song). 3,562 songs had a preview and complete descriptors; this is the *audio subset*. The same search was later run on every song (35,370 found, Appendix J). From the previews we extracted 31 spectral/rhythmic descriptors ("FFT") and 185 richer librosa [24] descriptors (20 MFCCs and their first and second derivatives, chroma, tonnetz, spectral contrast, centroid, bandwidth, roll-off, flatness, zero-crossing rate, RMS and onset/tempogram statistics, each summarised by mean and standard deviation).

ISMIR 2019. The release of Zangerle et al. [4] provides Essentia low- and high-level descriptors for 5,832 Hot 100 hits and 89,235 non-hits from the Million Song Dataset, in two files (`msd_bb_matches.csv`, `msd_bb_non_matches.csv`) that give for each track its MSD identifier, artist, title and release year (from the MSD metadata). Tracks with a missing or zero year are removed. On ISMIR the artist name is only lower-cased and trimmed (no accent or featuring removal). We flatten all numeric descriptors of the Essentia JSON files: numbers are kept, lists of numbers are summarised by their mean, while booleans, strings, lists of lists, the whole `metadata` block (bit rate, codec, ...) and the beat positions in seconds are skipped; we then keep the columns present for more than 95 % of the tracks (computed over all tracks with descriptors: the hits and the whole non-hit pool), giving 592, drop every track with a missing value in one of them (no imputation), keep the 5,831 hits with complete descriptors and, like the original study, draw an equal number of non-hits (protocol A, 11,662 songs). The year-matched sample of protocol B also has 11,662 songs. Hits have a median release year of 1987 and non-hits of 2001, a fact that matters below.

SpotGenTrack. 101,909 Spotify tracks with 13 descriptors, 207 pre-extracted low-level audio features and a popularity score [5]; we label tracks with popularity ≥ 55 as hits (19.4 %). Because the classes are imbalanced, we report ROC-AUC alongside accuracy for this dataset.

A fourth candidate, a Billboard year-end chart file (2000–2023) augmented with Spotify features, was discarded: 2,911 of its 3,397 rows have no audio descriptors.

Appendix E
Main result: more detail

TABLE IX
Cross-validated accuracy per decade: the models of Table III, trained on all decades in each of the 25 folds, scored separately on the test songs of each decade (Random Forest).

Features	60s	70s	80s	90s	00s	10s
Audio (13)	.743	.750	.770	.806	.836	.816
+ decade	.764	.763	.794	.840	.851	.833
+ artist	.763	.784	.812	.831	.860	.842
+ decade + artist	**.788**	**.796**	**.836**	**.864**	**.873**	**.868**

The improvement holds in every decade (Table IX), from +3.7 pts in the 2000s to +6.6 pts in the 1980s. Since each decade file is balanced, the decade indicator carries no information on its own; its gain can only come from interactions with the audio descriptors, i.e. from era-specific decision rules.

TABLE X
Robustness to the classifier (5-fold CV with one repeat, 5×1, corrected t-test; all 5 folds improve).

Classifier	Audio (13)	+ decade + artist	Δ	p
Random Forest [22]	.780	.832	+5.2	$2{\cdot}10^{-5}$
XGBoost [23]	.788	**.845**	+5.7	$3{\cdot}10^{-7}$
Logistic regression	.727	.782	+5.5	$4{\cdot}10^{-5}$
k-NN ($k = 25$)	.750	.796	+4.6	$8{\cdot}10^{-6}$
SVM (RBF)	.773	.828	+5.5	$3{\cdot}10^{-5}$

Robustness to the classifier. The gain is of the same size for five classifiers of very different nature (Table X), so it comes from the features rather than from the model. XGBoost is the strongest on average, but on the canonical split it ties with the frozen Random Forest (0.820 vs 0.822, McNemar $p = 0.87$). TreeSHAP attributions of the XGBoost model rank artist history first (mean $|\phi| =$

1.08), the six decade indicators second (0.86 summed) and instrumentalness third (0.73), ahead of every other descriptor.

Future decades. To test whether the gain survives a temporal shift, we train on all earlier decades and test on a later one, with no song from the test decade seen in training: 1960s–80s → 1990s, 1960s–90s → 2000s, 1960s–2000s → 2010s (we start at the 1990s so that the training set always spans at least three decades). Artist history improves the Random Forest by +2.5 pts on the 1990s (0.736 → 0.761), +1.9 pts on the 2000s (0.805 → 0.824) and +3.4 pts on the 2010s (0.780 → 0.814; McNemar p between 10^{-7} and 10^{-21}), with similar gains for XGBoost. The decade indicator cannot be used here, since the test decade never appears in training.

Appendix F
Where the accuracy comes from: PCA and everything together

Which descriptors matter: PCA. To see which of the 13 Spotify descriptors carry the signal, we ran a principal component analysis on the standardised descriptors (Fig. 11). The first component (23.6 % of the variance) is an *intensity* axis (energy +0.48, loudness +0.47, acousticness −0.43) and is also the one that best separates hits (ROC-AUC 0.70 on its own). The second (11.1 %) is a *mood* axis (danceability +0.48, valence +0.44, duration −0.36; AUC 0.62). The next four components together explain another 33 % of the variance but separate hits no better than chance (AUC 0.50–0.52), and eight components are needed to reach 81 %. Variance and usefulness therefore do not coincide. Among single descriptors, the most separating are instrumentalness (AUC 0.74, lower in hits), danceability (0.70, higher), loudness (0.64) and valence (0.64); key, duration and tempo are close to chance (0.51–0.52). Replacing the descriptors by their components only loses accuracy: the frozen Random Forest reaches 0.575 with one component, 0.710 with six and 0.765 with all 13, against 0.782 for the raw descriptors (−1.7 pts even though no information is removed, 0/25 folds, $p = 2 \times 10^{-8}$). Trees split on one input at a time, and rotating the axes mixes a sparse, highly informative descriptor (instrumentalness) into components shared with uninformative ones. We therefore keep the raw descriptors; PCA is used only for the 185-dimensional librosa representation (Appendix J).

Training on everything at once. One might simply put every input we built into one model. We did this on the 35,370 songs that have a preview (needed for FFT), with the same frozen Random Forest and 5×5 CV (Table XI). The full model reaches 0.883, 5.8 points above the final model (0.825 on these songs). That gain does not come from music. Removing the song title alone takes it down to 0.843 (−4.0 pts, $p = 3 \times 10^{-19}$), and the title is an artefact of how the non-hits were collected (Appendix J). The remaining +1.8 pts over the final model must come, by elimination, from FFT and the market-trend features, the only other blocks that matter (removing either one costs −0.4 and −0.3 pts; title length and `chorus_hit`/`sections` change nothing; the two overlap, so each costs less alone), whose gains vanish under the genre control and carry only the era (Appendix J and Sec. V-H). Once FFT and trend are in, the decade indicators become redundant (−0.2 pts, $p = 0.11$): all three describe the era. Artist history remains the input whose removal costs the most among the legitimate ones (−1.9 pts, $p = 2 \times 10^{-11}$), ahead of the 13 descriptors (−1.2 pts), while `chorus_hit`/`sections` and title length change nothing. Training on everything therefore gives a higher number on this benchmark by learning how it was built. The model we keep is the one whose inputs survive the controls: the 20 inputs of the final model.

TABLE XI
Training on every input at once (35,370 songs with a preview, frozen Random Forest, 5×5 CV). Left: one block removed from "everything" (Δ vs everything). Right: the block alone. *Decade alone scores .532 here, not the .503 of the full benchmark, because these songs are 53.2 % hits: the decade carries no information and the model predicts the majority class.

Model	Acc.	AUC	Δ (p)
Final model (Spotify + decade + artist)	.825	.911	–
Everything (8 blocks)	**.883**	.952	+5.8 ($3{\cdot}10^{-21}$) vs final
Everything except song title	.843	.925	+1.8 ($6{\cdot}10^{-10}$) vs final
Remove one block from everything			
− song title	.843	.925	−4.0 ($3{\cdot}10^{-19}$)
− artist history	.864	.933	−1.9 ($2{\cdot}10^{-11}$)
− Spotify (13)	.871	.944	−1.2 ($5{\cdot}10^{-8}$)
− FFT (31)	.879	.949	−0.4 (.01)
− market trend (2)	.879	.949	−0.3 (.01)
− decade (6)	.881	.950	−0.2 (.11)
− title length	.883	.951	−0.0 (.93)
− chorus_hit + sections	.883	.952	+0.0 (.75)
One block alone			
Spotify (13) / FFT (31)	.777 / .773	.849 / .847	–
artist history / song title	.648 / .660	.669 / .740	–
market trend / title length	.629 / .546	.672 / .545	–
chorus_hit + sections / decade	.543 / .532*	.563 / .516	–

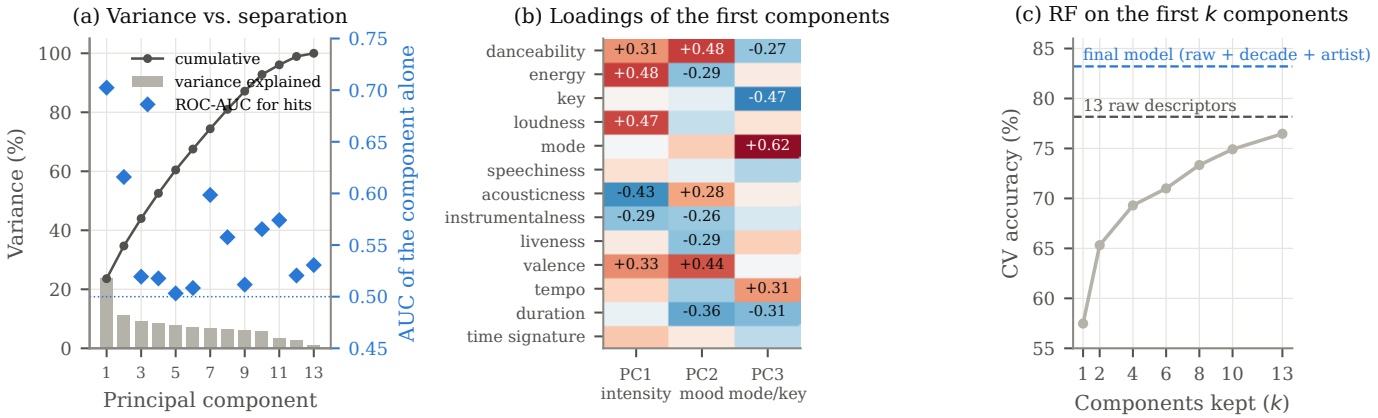


Fig. 11. PCA of the 13 Spotify descriptors (40,560 songs). (a) Variance explained by each component (bars), cumulative (line), and how well each component alone separates hits from non-hits (ROC-AUC, right axis; dotted line = chance). (b) Loadings of the first three components (values below 0.25 in absolute value are not printed). (c) Cross-validated accuracy of the frozen Random Forest trained on the first k components, against the 13 raw descriptors and the final model.

Appendix G
Head-to-head with Du: table

TABLE XII
Head-to-head with Du [10] using their protocol (one model per decade, 13 audio + `chorus_hit` + `sections`, 70/30 split) over 50 random splits: mean accuracy (std). RF: Random Forest; LR: logistic regression; cfg.: configuration. "Wins" counts splits where adding artist history helps; p: corrected t-test over the $J = 50$ paired splits, $n_{\text{TEST}}/n_{\text{TRAIN}} = 3/7$.

Dec.	Model	Published	Re-impl.	+ artist	Wins / p
00s	RF (their cfg.)	.84	.846 (.009)	**.867** (.009)	50 / $9{\cdot}10^{-8}$
	LR (their cfg.)	.82	.821 (.007)	**.848** (.007)	50 / $2{\cdot}10^{-9}$
	RF (frozen)	–	.852 (.008)	**.875** (.006)	50 / 10^{-7}
10s	RF (their cfg.)	.82	.829 (.008)	**.861** (.008)	50 / $4{\cdot}10^{-11}$
	LR (their cfg.)	.79	.807 (.009)	**.835** (.008)	50 / $6{\cdot}10^{-6}$
	RF (frozen)	–	.841 (.008)	**.873** (.008)	50 / $9{\cdot}10^{-10}$

Appendix H
External validation: more detail

TABLE XIII
SpotGenTrack (101,909 tracks, 19.4 % hits; always predicting "non-hit" scores 0.806). 5-fold CV, corrected t-test vs the 13 descriptors.

Features	Acc. [95 % CI]	ROC-AUC	p (acc.)
Spotify descriptors (13)	.812 [.811,.813]	.690	–
+ decade	.812 [.811,.813]	.712	.66
+ artist	.823 [.821,.826]	.762	10^{-3}
+ decade + artist	**.825** [.823,.827]	**.777**	$3{\cdot}10^{-4}$
+ artist + low-level audio	.825 [.823,.828]	.758	$2{\cdot}10^{-4}$

SpotGenTrack. Here the label is Spotify popularity rather than a chart, and 80.6 % of tracks are negatives, so accuracy is dominated by the majority class: the 13 descriptors score 0.812, barely above always predicting "non-hit". Ranking quality tells the real story (Table XIII):

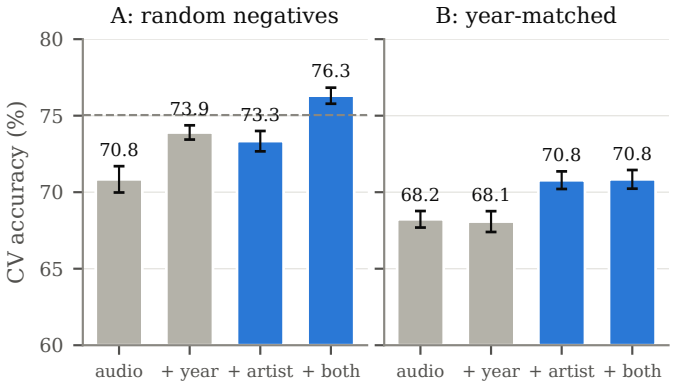


Fig. 12. ISMIR 2019 data, cross-validated accuracy with 95 % CI. Gray: without artist history; blue: with it ("both" = year + artist). Dashed line in A: best published accuracy [4]. Matching years (B) removes the gain of release year but not that of artist history.

artist history raises ROC-AUC from 0.690 to 0.762 (+7.2 points), and with decade to 0.777. On the hit class, the 13 descriptors find only 8.3 % of the hits (F1 0.147); with decade and artist history, recall rises to 25.2 % at unchanged precision (0.62) and F1 to 0.358 (5/5 folds, $p = 7 \times 10^{-7}$), while the 207 low-level audio features add nothing on top of artist history. A single 75/25 split gives the same picture (accuracy 0.810 → 0.822 and ROC-AUC 0.688 → 0.760 with artist history, McNemar $p = 6 \times 10^{-13}$).

Appendix I
Unseen artists and cold start: table

Artist-grouped cross-validation. With random folds, songs by the same artist can fall in both the training and the test part, so a model could recognise an artist from the audio. We therefore repeat every protocol with stratified group k-fold, grouping songs by normalised artist so that no test artist is seen in training (Table XIV,

TABLE XIV

ROBUSTNESS. TOP: CROSS-VALIDATED ACCURACY WITH RANDOM STRATIFIED FOLDS VS FOLDS GROUPED BY ARTIST (NO TEST ARTIST SEEN IN TRAINING). ERA = DECADE (HIT PREDICTOR) OR RELEASE YEAR (ISMIR). BOTTOM: GAIN OVER AUDIO IN GROUPED CV, ON SONGS WHOSE ARTIST HAD NO EARLIER HIT (NEW) OR AT LEAST ONE (KNOWN). NEW/KNOWN SONGS: 33,438/7,122 (HIT PREDICTOR), 7,480/4,182 (ISMIR A), 7,269/4,393 (ISMIR B). THE ACCURACY ON EACH GROUP IS COMPUTED INSIDE EACH TEST FOLD; THE CORRECTED t-TEST USES THE PER-FOLD DIFFERENCES ON THAT GROUP AND THE RATIO $n_{\text{TEST}}/n_{\text{TRAIN}}$ OF THE WHOLE FOLD.

Data	Folds	Audio	+ era	+ artist	+ both
Hit Predictor	random	.782	.802	.810	.832
	by artist	.772	.792	.802	**.822**
ISMIR A	random	.708	.739	.733	.763
	by artist	.709	.739	.730	**.759**
ISMIR B	random	.682	.681	.708	.708
	by artist	.677	.678	**.705**	**.705**

	Hits (%)	New artists: Δ (p)		Known: Δ (p)
	new/known	+ era	+ artist	+ artist
Hit Predictor	39 / 97	+2.2 ($2{\cdot}10^{-9}$)	+0.4 (.17)	+15.0 ($3{\cdot}10^{-14}$)
ISMIR A	36 / 75	+3.4 ($6{\cdot}10^{-5}$)	+0.8 (.08)	+4.4 (.003)
ISMIR B	37 / 72	+0.1 (.79)	+1.4 (.002)	+5.3 ($8{\cdot}10^{-5}$)

top). Accuracies drop by up to one point, a small artist-identity effect in the audio descriptors, but the gains remain: +5.0 pts for decade and artist on the Hit Predictor (unchanged, 25/25 folds, $p = 10^{-13}$) and +2.9 pts for artist history in the year-matched ISMIR protocol (10/10, $p = 9 \times 10^{-7}$). In protocol A the gains lose at most half a point, and audio, year and artist still reach 0.759 [0.752, 0.766], above the published 75.04 %.

Cold start. Table XIV (bottom) splits the grouped test folds by whether the artist had an earlier hit. On the Hit Predictor, artist history does not help new artists (+0.4 pts, $p = 0.17$): its whole gain comes from the 17.6 % of songs by established artists, 96.6 % of which are hits, which quantifies the benchmark effect discussed in Sec. VI. The decade, in contrast, helps new artists (+2.2 pts, $p = 2 \times 10^{-9}$). On ISMIR, where songs by established artists are hits only 72–75 % of the time, artist history also tends to help new artists in the year-matched protocol (+1.4 pts, $p = 0.002$, 10/10 folds; above the Bonferroni threshold, so indicative only), perhaps because an empty history is itself informative when negatives are not selected by artist. On ISMIR, "new" means no earlier hit among the dataset's own hits (the reference set there), not in the whole Hot 100.

APPENDIX J
AUDIT AND FURTHER EXPERIMENTS

Titles. A title model trained out-of-fold adds +6.7 pts, more than any musical feature. Its largest weights show why. On the non-hit side: *remastered, feat, live, version, remix, mix, edit, demo, mono, medley*, genre words (*blues, metal, ambient, punk, rock, death, jazz, hardcore*) and non-English words (*remasterizado, de, te, en*); on the hit side: ordinary words (*nights, you, party, kiss, love, sunshine*).

TABLE XV

AUDIT OF CANDIDATE GAINS. SINGLE SPLIT (MCNEMAR) VS 5×5 CROSS-VALIDATION (CORRECTED t-TEST). TOP: FULL BENCHMARK (SINGLE SPLIT = THE CANONICAL SPLIT, 2,000 TEST SONGS). BOTTOM: 3,562-SONG AUDIO SUBSET (SINGLE SPLIT = ITS STRATIFIED 75/25 SPLIT, 891 TEST SONGS).

Added to audio	Split Δ (p)	CV Δ	CV p	Verdict
Song title (TF-IDF)	+6.9 ($4{\cdot}10^{-14}$)	+6.7	$4{\cdot}10^{-20}$	artefact
`chorus_hit+sections`	+0.6 (.32)	−0.0	.82	no effect
Title length	+0.05 (1.0)	+0.2	.28	no effect
Rich librosa (PCA 40)	+3.8 (.002)	+0.9	.15	not significant
FFT descriptors (31)	+4.4 (.001)	+2.3	.007	small
Decade	+1.1 (.27)	+1.0	.03	small
Artist	+3.4 (.010)	+3.8	$2{\cdot}10^{-6}$	robust
Artist + rich	+6.7 (10^{-6})	+4.9	10^{-8}	= artist
Decade + artist	+3.6 (.006)	+5.1	10^{-8}	robust

On the genre-matched sample of Appendix J (7,554 songs), 13.5 % of the non-hit titles contain " - " (a Spotify suffix such as "- Remastered 2011"), against 0.1 % of the hits; 5.7 % contain "remaster" (hits: 0), and 5.7 % contain non-ASCII letters (hits: 0.1 %). The hit titles have evidently been cleaned and the non-hit titles have not. We test this directly (`r8_future_work.py title`, 5×5 CV on the genre-matched sample). The title adds +4.8 pts to the 13 descriptors and +2.9 pts to the final model (25/25, $p = 2 \times 10^{-6}$), so genre explains only part of it. We then strip everything after " - ", anything in brackets and "feat./ft./featuring ..." (this changes 19.0 % of non-hit titles and 4.9 % of hit titles). The gain over the final model falls to +1.9 pts (23/25, $p = 0.002$), above the Bonferroni threshold. What remains is mostly genre and language vocabulary. We reject the title: its gain comes from how the two classes were written down and collected, not from anything that makes a song a hit.

Single split vs cross-validation. On the small audio subset, the 185 rich librosa descriptors appeared to add +3.8 pts with $p = 0.002$ on the subset's single 75/25 split, but the cross-validated gain is +0.9 pts and is not significant. The same holds for the choice of the PCA dimension (5 to 100 components are within noise) and for the knock-out of any descriptor family (all within ±0.1 pts). In a block ablation of the full subset model, only removing artist history causes a significant drop (−3.3 pts, $p = 10^{-5}$). The `chorus_hit` and `sections` fields, often suspected of leakage, add nothing.

FFT and the best audio-subset model. The 31 FFT descriptors are the only extra audio representation whose gain survives cross-validation, and it persists on top of artist history: audio + artist + FFT reaches 0.821 against 0.801 for audio + artist (+2.0 pts, 23/25 folds, $p = 0.008$). Adding decade to that model gives the best subset accuracy, 0.824, but the increment is not significant (+0.3 pts, $p = 0.50$), and decade adds nothing to FFT alone (0.785 with decade vs 0.786 without): the spectral descriptors already carry the era. Conversely, artist history still adds +3.5 pts on top of FFT ($p = 4 \times 10^{-5}$). Fig. 13

shows every configuration we evaluated on this subset.

FFT on the whole benchmark. We then retrieved Deezer previews for every song we could match: 35,370 of the 40,560 songs (87.2 %). Deezer finds hits more often than flops (94.0 % vs 80.6 %), so the covered set is 53.2 % hits and every comparison below uses the same 35,370 songs. With 5×5 cross-validation, adding FFT to the main model (audio + decade + artist) raises accuracy from 0.825 to 0.839 (+1.4 pts, 25/25 folds, $p = 4 \times 10^{-9}$; AUC 0.911 → 0.921). The whole gain comes from songs by new artists, 0.789 → 0.807 (+1.8 pts, 25/25, $p = 6 \times 10^{-9}$), while songs by known artists stay at 0.977 (−0.1 pts, n.s.). Unlike on the small subset, decade remains useful once FFT is present: audio + artist + FFT reaches only 0.829 (+0.4 pts over the main model, $p = 0.04$). FFT thus appears to address the cold-start weakness of artist history, but the benchmark's flops come from non-mainstream genres by construction, and spectral descriptors can recognise genre. Appendix J shows that this gain does not survive a genre control.

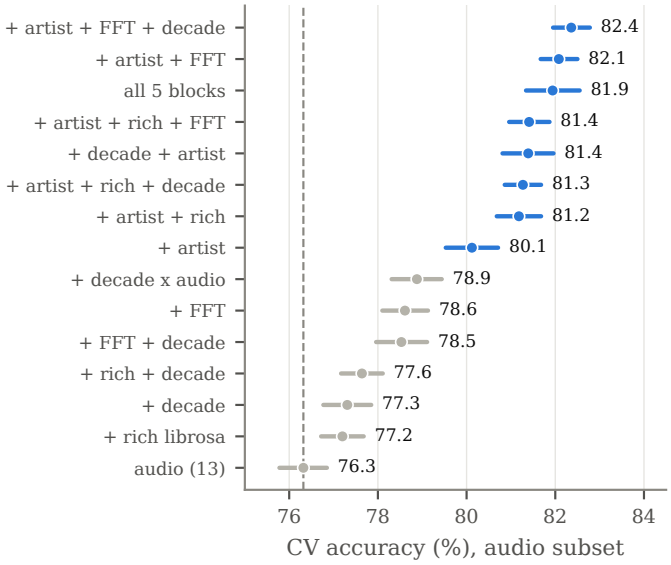


Fig. 13. Every configuration evaluated on the 3,562-song audio subset, 5×5 cross-validated accuracy with 95 % CI, sorted. Blue: with artist history; gray: without. Dashed line: the 13 Spotify descriptors alone. "rich": the 185 librosa descriptors (PCA 40); "decade x audio": each of the 13 descriptors multiplied by each of the 6 decade indicators (78 inputs); "all 5 blocks": audio + artist + decade + decade x audio + rich.

Genre control for FFT. We attach Spotify artist genres from the SpotGenTrack release [5] by normalised artist name (20,610 songs with both a preview and a genre) and map the tags to 16 families. The benchmark's construction is plainly visible: Latin, classical, jazz, "world" and metal artists account for 50 % of the flops and 1.9 % of the hits, while rock, pop, R&B, hip-hop and country account for 93 % of the hits. Genre is also partly audible: a Random Forest (200 trees, stratified shuffled 5-fold cross-validated predictions, seed 42) recovers the dominant family from the 13 Spotify descriptors with 47.4 % accuracy, from the 31 FFT descriptors with 49.9 % and from both with 53.3 % (majority class 26.1 %). Three results follow (Table XVI). (a) Genre families as features raise the main model from 0.828 to 0.928 (+10.1 pts, 25/25, $p = 8 \times 10^{-23}$): on this benchmark, knowing the artist's genre is worth more than any audio representation. (b) Once genre is known, FFT *lowers* accuracy (−1.1 pts, 0/25, $p = 5 \times 10^{-7}$), also for new artists (−1.4 pts). (c) On a genre-matched sample (as many hits as flops inside every family; 7,554 songs), FFT adds nothing to the main model (+0.1 pts, 13/25, $p = 0.84$; new artists +0.1 pts, $p = 0.80$). FFT still improves the 13 Spotify descriptors alone on that sample (+2.5 pts, 25/25, $p = 5 \times 10^{-4}$), but the main model, without FFT, still beats audio + FFT by 4.3 pts ($p = 2 \times 10^{-8}$), and decade and artist history add +6.7 pts to the 13 descriptors on that sample (0.722 → 0.790, 25/25, $p = 9 \times 10^{-13}$); each one alone also helps on that sample (decade +2.5 pts, $p = 8 \times 10^{-7}$; artist history +4.2 pts, $p = 2 \times 10^{-8}$; `r8_future_work.py decadegenre`). We conclude that the full-benchmark FFT gain is a genre shortcut, whereas the gain of artist history survives the genre control. Spotify assigns artist genres retrospectively, so we use them only as a control, never as a predictive feature.

TABLE XVI
GENRE CONTROL FOR FFT (5×5 CV). LEFT: SONGS WITH A PREVIEW AND A SPOTIFY GENRE. RIGHT: GENRE-MATCHED SAMPLE. ACCURACY OVERALL AND ON SONGS BY NEW ARTISTS.

	All (20,610)		Matched (7,554)	
Model	Acc.	new art.	Acc.	new art.
audio (13 descriptors)	–	–	.722	–
main (audio+decade+artist)	.828	.780	.790	.749
+ FFT	.841	.798	.791	.750
+ genre	.928	.911	–	–
+ genre + FFT	.917	.897	–	–
decade vs audio, Δ (*p*)		–		+2.5 ($8{\cdot}10^{-7}$)
artist vs audio, Δ (*p*)		–		+4.2 ($2{\cdot}10^{-8}$)
main vs audio, Δ (*p*)		–		+6.7 ($9{\cdot}10^{-13}$)
FFT vs main, Δ (*p*)		+1.3 ($1.1{\cdot}10^{-4}$)		+0.1 (.84)
FFT vs main+genre, Δ (*p*)		−1.1 ($5{\cdot}10^{-7}$)		–

Artist resolution. The 52.1 % match rate is not a matching problem: 99.99 % of the hits have an artist found in the chart history (by construction, at the latest in their own decade), against 5.4 % of the flops, and only 2.7 % of the flops have an artist who charted up to and including their own decade (only about 1.2 % charted in an *earlier* decade, i.e. have $h(s) > 0$). The remaining ≈1.5 % of flops have an artist whose normalised name has a Hot 100 entry in the flop's own decade, although the dataset's rule excludes such artists: these are homonyms merged by our name matching, or chart entries missing from the dataset's own hit list; they do not enter $h(s)$, which counts earlier decades only. The unmatched songs are flops whose artists never charted. Two alternatives give no gain: matching the full credited name before the primary

artist ("Simon & Garfunkel" rather than "Simon") lowers accuracy (0.827 vs 0.832, −0.5 pts, $p = 4 \times 10^{-4}$), and crediting featured appearances changes nothing (+0.1 pts, $p = 0.39$). Canonical identifiers would avoid homonyms but cannot raise the match rate.

How fast does a model age? We train on a single decade and test on every later one (Table XVII, songs with a preview). Audio-only accuracy falls from 0.809 within a decade to 0.765 one decade later and 0.689 three decades later. Artist history slows the decay, and its advantage grows with the gap (+2.8 pts within a decade, +5.1 pts three decades later). FFT behaves in the opposite way: it helps within a decade (+2.2 pts) but hurts from three decades on, consistent with descriptors that encode era-specific production and genre rather than lasting appeal. The cells average 6 to 1 train/test pairs and are not tested for significance.

TABLE XVII
MODEL AGEING: TRAIN ON ONE DECADE, TEST k DECADES LATER (FROZEN RANDOM FOREST OF THE MAIN MODEL, ONE FIT PER PAIR, 35,370 SONGS WITH A PREVIEW; $k = 0$: 5-FOLD CV INSIDE EACH DECADE). MEAN ACCURACY OVER THE AVAILABLE DECADE PAIRS.

k	pairs	audio	+ artist	+ FFT	+ artist + FFT
0	6	.809	.837	.831	**.859**
1	5	.765	.796	.782	**.810**
2	4	.728	**.775**	.736	**.775**
3	3	.689	**.740**	.673	.728
4	2	.666	**.734**	.661	.718
5	1	.664	**.754**	.599	.706

Lyrics on SpotGenTrack. SpotGenTrack provides six lexical descriptors of the lyrics (syllables per word, words per sentence, sentence count, word count, sentence similarity, vocabulary richness) for 79,420 tracks with lyrics (21.5 % hits; 5×1 CV as in Sec. V-D). They improve the 13 Spotify descriptors (AUC 0.678 → 0.729, $p = 3 \times 10^{-4}$) and still add on top of decade and artist history (accuracy 0.808 → 0.814, 5/5 folds, $p = 0.002$; AUC 0.774 → 0.797, $p = 2 \times 10^{-5}$). The gain is modest in accuracy because of the class imbalance, and such a small gain measured on five folds does not survive the Bonferroni correction, so we tested lyrics again on a larger, balanced dataset.

APPENDIX K
LYRICS ON ISMIR: MORE DETAIL

ISMIR songs are Million Song Dataset tracks (joined to the lyrics by their MSD track identifier), for which the official musiXmatch release [26] gives the lyrics of 237,662 tracks as word counts over the 5,000 most frequent (stemmed) words. Coverage is itself a shortcut: 56 % of the hits have lyrics against 36 % of the non-hit pool, so "has lyrics" would predict the label. We therefore keep only songs *with* lyrics on both sides and redraw balanced samples under protocols A and B. A second shortcut is language: 97.5 % of the hits but only 80.5 % of the non-hits are mainly in English (at least 25 % of their words among the 29 most common English words). We therefore also repeat every test on English songs only: the non-English songs are removed first, and the balanced (protocol A) or year-matched (protocol B) sample is then drawn again from what remains. Two lyrics inputs are added to the final ISMIR model (audio + year + artist history): eight lexical features (log number of words and of distinct words, richness, repetition, share of *I/me/my*, *you/your*, *love* and of common English words), and one bag-of-words score, the out-of-fold probability of a TF-IDF logistic regression trained inside each training fold, as for titles.

TABLE XVIII
LYRICS ON ISMIR 2019 (SONGS WITH LYRICS ONLY; RANDOM FOREST, 5×2 CV, CORRECTED t-TEST VS THE FINAL MODEL OF THE SAME ROW; ALL ROWS 10/10 FOLDS IMPROVED). UNSEEN: NO ARTIST IN BOTH TRAINING AND TEST FOLDS.

Sample	Songs	Final	+ lyrics	Δ (p)	AUC
A: random	6,526	.728	**.775**	+4.7 ($3{\cdot}10^{-4}$)	.798→.851
B: year-matched	6,478	.676	**.727**	+5.1 ($2{\cdot}10^{-5}$)	.732→.802
A, English	6,364	.718	**.745**	+2.7 ($6{\cdot}10^{-4}$)	.788→.820
B, English	6,294	.658	**.708**	+4.9 ($1.1{\cdot}10^{-4}$)	.720→.772
A, English, unseen	6,364	.712	**.742**	+3.0 ($6{\cdot}10^{-4}$)	.785→.817
B, English, unseen	6,294	.656	**.700**	+4.4 ($3{\cdot}10^{-5}$)	.717→.764

Lyrics alone (the 8 lexical features + bag-of-words score, no audio, year or artist; B, year-matched): .702, above the final model without lyrics (.676). Artist history uses all ISMIR hits as reference set.

Table XVIII and Fig. 9a show that lyrics add +4.7 points in protocol A and +5.1 in the year-matched protocol B, in all 10 folds. The gain shrinks but remains when only English songs are kept (+2.7 and +4.9), and it remains when, in addition, no artist appears in both training and test folds (+3.0 and +4.4, $p = 3 \times 10^{-5}$; AUC +0.047, $p = 10^{-6}$), so it does not come from recognising the vocabulary of artists seen in training. In the strictest setting (year-matched, English, unseen artists) the gain survives the Bonferroni correction ($p = 3 \times 10^{-5} < 1.25 \times 10^{-4}$), as do the other protocol-B rows; the three protocol-A rows ($p = 3 \times 10^{-4}$ to 6×10^{-4}) do not, and are indicative only. In the year-matched protocol, lyrics alone (0.702) are more accurate than audio, year and artist history together (0.676). Fig. 9b shows what the model reads. Hit lyrics favour *love* (by far the largest weight), vocables (*la*, *oh*, *ooh*, *yeah*, *hey*) and direct address (*need*, *girl*) and night-out themes (*night*). Non-hit lyrics favour darker or narrative words (*kill*, *hate*, *crime*, *mind*, *their*, *the*). On average, hit lyrics are slightly longer and use a smaller share of distinct words (richness 0.34 vs 0.42), use *love* twice as often (1.3 % vs 0.65 % of words) and use *you* more (5.5 % vs 4.6 %). These words point to themes and genres (romantic pop against metal, punk or hardcore rap) as much as to style. On ISMIR this is a real difference between chart songs and the rest of the catalogue rather than a construction rule, since the negatives are drawn at random. But it means that lyrics, like genre, tell *mainstream* songs from the rest more than they measure the appeal of an individual song. We could not test lyrics on the Hit Predictor, whose songs have no Million Song Dataset identifier.

Appendix L
Time series: chart components, market trend, trajectory table

The chart as a time series. The weekly Hot 100 (3,545 charts, 1958–2026) also forms time series. A *debut* is the first week a (title, performer) pair appears anywhere in the chart history (re-entries are not debuts). The monthly mean number of debuts per chart week (1959–2025) is mostly trend: an STL decomposition [28] gives a trend strength of 0.72 and a seasonal strength of 0.11 [29]. Debuts fall from 11.0 per week in 1959 to 5.5 in 2001 and return to 10.9 in 2025, after the chart started counting streams; the largest residual is a rule change (60 debuts in the chart of 5 December 1998, when songs not released as singles became eligible). The weekly number of holiday songs on the chart is almost purely seasonal: its periodogram peaks at 52.1, 26.1, 17.4 and 13.0 weeks, and a Fourier regression fitted on the streaming era (charts from July 2018 on, a recent window in which streaming dominates the chart; target $\ln(1+\text{count})$, a linear trend plus $K = 20$ harmonics) explains 94 % of its variance. Forecasting monthly debuts 12 months ahead under rolling-origin evaluation (11 origins, 2015–2025), ARIMA$(1,1,2)$ [30] and ARIMA with Fourier terms (MASE 1.82) and damped Holt-Winters (1.85) are best and indistinguishable (ARIMA + Fourier vs ARIMA, $p = 0.97$); they beat the seasonal naive forecast (2.32) at 10 of 11 origins ($p = 0.006$). Both p-values come from a two-sided paired t-test on the 11 per-origin MASE values (the 12-month test windows of different origins do not overlap). A seasonal ARIMA (1.89) and gradient boosting or random forests on lag and Fourier features (1.94, 1.99) do worse: the series has little seasonality, and 800 points leave tree models little to learn.

Before release: market trends. We describe each Hit Predictor song by the distance of its standardised audio descriptors to the centroid of the previous decade's hits, and by its projection on the direction in which that centroid moved. On top of the main model these two features add +0.07 pts ($p = 0.66$); the 13 per-feature deviations add +0.46 pts (23/25, $p = 0.006$). On audio alone the trend features add +1.8 pts ($p = 7 \times 10^{-10}$), less than the decade alone (+2.0 pts): they carry era information, which the main model already has.

TABLE XIX
Predicting a top-10 peak from a song's first k weeks on the Hot 100 (31,925 songs, 16.4 % top-10; Random Forest, 5×3 CV). Values are ROC-AUC except the F1 column (F1 on the top-10 class of the weeks + context model, threshold 0.5). Context: year, month and the artist's earlier chart history. Last column: songs not yet in the top 10 after k weeks (ROC-AUC).

	All songs				Not yet top 10
k	weeks	+ context	F1	context only	+ context
1	.700	.750	.401	.593	.720
2	.796	.850	.531	.593	.828
3	.845	**.892**	.610	.593	.870
4	.891	**.917**	.674	.593	.894

Appendix M
Single splits and multiple comparisons

Single splits mislead. Several conclusions of our own earlier single-split analysis did not survive cross-validation. With a few thousand songs, an 891-song test set has a standard error of about 1.4 pts, so single-split gains of 3–4 pts on subsets are fragile even when McNemar's test is significant on that split. **Multiple comparisons.** Each p-value printed in the paper is the raw value of its own test. Bonferroni divides 0.05 by the number of tests in the family. We take as family every p-value printed in a table or in the text that compares two configurations, counting a test again each time it is printed (the survival ranking, Tables III, X, XII, IV, XIII, XIV, XV, XI, XVI, XVIII, the ablation and the values given only in the text), plus the exploratory tests we ran but do not print one by one: the PCA dimensions of the librosa block, the knock-out of each librosa descriptor family and the configurations of Fig. 13 (on the audio subset: 7 PCA settings, 8 family knock-outs and 13 configurations, i.e. fewer than 30 tests). Rather than rely on an exact count, we use a deliberately generous family of 400, well above that total; the threshold is $p < 0.05/400 = 1.25 \times 10^{-4}$.

Every result our conclusions rest on passes it: the cross-validated gains of Tables III and X ($p \le 4 \times 10^{-5}$), the head-to-head of Table XII ($\le 6 \times 10^{-6}$), the temporal hold-out ($\le 10^{-7}$), the ISMIR gains of artist history in both protocols ($\le 3 \times 10^{-5}$), the artist-grouped gains, the genre-matched gains of decade (8×10^{-7}), artist history (2×10^{-8}) and both (9×10^{-13}), and the year-matched lyrics tests (protocol B: 2×10^{-5}; English only: 1.1×10^{-4}; English, unseen artists: 3×10^{-5}). The full-benchmark FFT gain on top of the main model also passes ($p = 4 \times 10^{-9}$) but is explained by genre (Appendix J); the genre-control results themselves ($p \le 5 \times 10^{-7}$) pass as well, and so do the early-trajectory gains of Table XIX ($p < 10^{-6}$).

Results with $1.25 \times 10^{-4} \le p < 0.05$ are indicative only: on the 2,000-song canonical split, decade alone (3×10^{-4}) and artist history alone (6×10^{-4}); the decade and FFT gains on the 3,562-song audio subset (including FFT on top of artist history) and most single-split gains on that

subset; audio + artist + FFT against the main model on the whole benchmark ($p = 0.04$); the per-feature market-trend deviations ($p = 0.006$); the forecasting comparison of Sec. V-H ($p = 0.006$, 11 origins); the lyrics gains on SpotGenTrack (5×1 CV, $p = 0.002$) and the three protocol-A lyrics rows of ISMIR (3×10^{-4} to 6×10^{-4}); the SpotGenTrack gains of artist history (10^{-3}) and of decade + artist (3×10^{-4}), which are in any case upper bounds (popularity is measured today, Sec. VI); the release-year gain on ISMIR A (2×10^{-4}), a shortcut anyway; and the ISMIR cold-start gains of artist history for known artists in protocol A and new artists in protocol B.

Appendix N
Which script produces what

TABLE XX
Scripts in `repro/` and the results they produce (`paper_results/R*_*.csv`).

Script	Results
`r1_kaggle_core.py`	main result, per decade, titles, chorus_hit (Tables III, IX)
`r2_models_time.py`	five classifiers, SHAP, future decades (Table X)
`r3_audio_subset.py`	audio subset, librosa, FFT (Table XV, Fig. 13)
`r4_external.py`	ISMIR, SpotGenTrack (Tables IV, XIII)
`r5_tilburg.py`	head-to-head with Du (Table XII)
`r6_robustness.py`	unseen artists, cold start (Table XIV)
`r7_fft_full.py`	FFT on the whole benchmark
`r8_future_work.py`	artist matching, genre control, title control, own-title check, decay, SpotGenTrack lyrics (Tables XVI, XVII)
`r9_metrics.py`	precision, recall, F1
`r10_timeseries.py`	market trend, early trajectories (Table XIX)
`r11_lyrics_ismir.py`	lyrics on ISMIR (Table XVIII, Fig. 9)
`r12_report.py`	PCA, ablation, everything together, trajectory figure (Sec. V-B, Fig. 10)
`r13_explain.py`	worked example and decade medians (Figs. 2, 3)
`make_figures.py`	all data figures
`timeseries/ build_notebook.py`	STL, periodogram, Fourier, ARIMA forecasting (Appendix O)

Appendix O
Time-series details

The chart analysis of Sec. V-H is in a notebook (`timeseries/time_series_hot100.ipynb`) that follows the standard sequence of a time-series study [29]: plot the series, decompose it into trend, seasonality and residual, look for cycles with the periodogram, model the cycles with Fourier terms, test stationarity (ADF, KPSS) and autocorrelation (ACF, PACF, Ljung–Box), and compare forecasting methods by rolling origin. Test settings (statsmodels defaults otherwise; computed on the whole 1959–2025 series): ADF with the lag order chosen by AIC; KPSS around a constant with automatic lag; Ljung–Box at lags 12 and 24. On the monthly debut series, ADF does not reject a unit root ($p = 0.51$) and KPSS rejects stationarity ($p = 0.01$); after one difference ADF rejects ($p < 0.001$) and KPSS does not ($p \geq 0.10$), hence $d = 1$ (fixed from the whole series, forecast windows included; the orders p, q are chosen on pre-origin data only). The STL residual is close to white noise (Ljung–Box does not reject at lags 12 and 24).

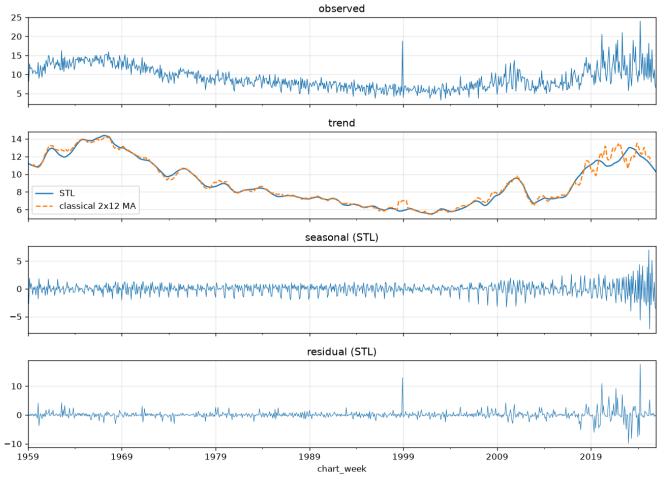


Fig. 14. STL decomposition of the monthly mean number of debuts per chart week (1959–2025). The trend dominates (strength 0.72); seasonality is weak (0.11). The spike of December 1998 is the rule change that admitted songs not released as singles.

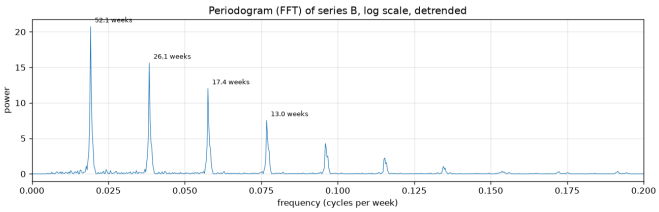


Fig. 15. Periodogram (FFT) of the weekly number of holiday songs on the chart ("series B" in the plot; computed on ln(1 + count) after removing the trend; titles containing Christmas, Santa, Jingle, Sleigh, Rudolph, Mistletoe, Navidad, Winter Wonderland, Holly Jolly, Silent Night or Snowman): peaks at one year and its harmonics (26.1, 17.4, 13.0 weeks), which a Fourier regression turns into the seasonal pattern.

ARIMA in one paragraph. ARIMA(p, d, q) models a series by differencing it d times until it has no trend, then explaining each value from its own p previous values (autoregression) and from the q previous forecast errors (moving average). We chose $d = 1$ from the stationarity tests and (p, q) by AIC over $p, q \leq 2$, on the data before

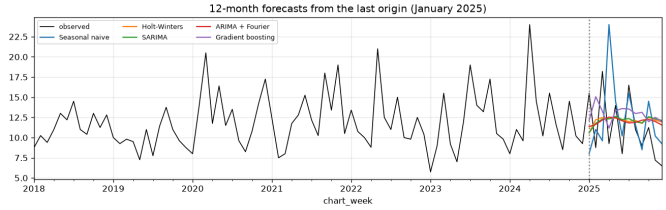


Fig. 16. Twelve-month forecasts of monthly debuts from the last origin (January 2025).

TABLE XXI
FORECASTING MONTHLY DEBUTS 12 MONTHS AHEAD, ROLLING ORIGIN (11 ORIGINS, 2015–2025). MASE: MEAN ABSOLUTE ERROR DIVIDED BY THAT OF THE IN-SAMPLE SEASONAL NAIVE FORECAST (LOWER IS BETTER).

Method	MASE
ARIMA$(1,1,2)$	**1.82**
ARIMA$(1,1,2)$ + Fourier terms	**1.82**
Damped Holt–Winters (ETS)	1.85
Seasonal ARIMA	1.89
Gradient boosting, lags + Fourier	1.94
Random forest, lags + Fourier	1.99
Seasonal naive	2.32

the first origin only. Adding Fourier terms as regressors captures yearly seasonality with few parameters.

Forecasting settings. Series: monthly mean number of debuts per chart week. Rolling origin: 11 origins, every January from 2015 to 2025; at each origin the models are fitted on all earlier months and forecast the next 12. STL: period 12, robust fitting. ARIMA$(1,1,2)$; ARIMA + Fourier: the same with $K = 2$ yearly harmonics as regressors; seasonal ARIMA: $(1,1,2)(P,0,Q)_{12}$ with $P, Q \leq 1$ chosen by AIC; ETS: additive damped trend and additive seasonality of period 12. Gradient boosting (300 trees, depth 3, learning rate 0.03, subsample 0.8) and random forest (300 trees, at least 5 samples per leaf) predict the monthly change $y_t - y_{t-1}$ from its 12 lags, the last level, the 12-month rolling mean and $K = 2$ Fourier terms of the month, and forecast 12 months recursively (each prediction is fed back as a lag); seed 42. MASE scales the mean absolute error by that of the seasonal naive forecast on the training months.

APPENDIX P
GLOSSARY

Bag of words / TF-IDF. A text is represented by how often each word occurs, ignoring order. TF-IDF weights a word by its (sublinear) frequency in the text, $1 + \ln \mathrm{tf}$, times $\ln \frac{1+N}{1+\mathrm{df}} + 1$, where N is the number of texts and df the number containing the word (the scikit-learn default), so that words found everywhere count little; each text vector is then scaled to length 1. A logistic regression on these weights gives a hit probability.

Bootstrap sample. A sample of n songs drawn *with replacement* from n songs; about 63 % of them appear at least once. Each tree of a Random Forest is grown on its own bootstrap sample.

ACF, PACF. Autocorrelation function: correlation of a series with itself ℓ steps earlier, for each lag ℓ; the partial version removes the effect of the shorter lags.

ADF, KPSS. Two stationarity tests (a series is *stationary* if its mean and variance do not drift). ADF assumes a unit root (the series behaves like a random walk, each value being the previous one plus a shock, so it is non-stationary) and tries to reject it; KPSS assumes stationarity and tries to reject it.

AIC. Akaike information criterion: goodness of fit penalised by the number of parameters; lower is better.

Anti-conservative. A test is anti-conservative when it gives p-values that are too small, i.e. it declares "significant" more often than it should.

Chroma, tonnetz. Energy of each of the 12 pitch classes (C, C#, ..., B) in a frame, computed from a short-time Fourier transform (STFT chroma) or from a constant-Q transform, whose frequency bins are spaced like musical notes (CQT chroma); tonnetz projects chroma on harmonic relations (fifths, thirds) and is computed on the *harmonic part* of the signal, obtained by separating sustained (harmonic) from percussive sounds.

ISMIR. International Society for Music Information Retrieval; "ISMIR 2019" is the dataset released with Zangerle et al.'s paper at its 2019 conference.

Essentia. An open-source library for audio analysis; the ISMIR release provides its low-level (spectral, rhythmic, tonal) and high-level (mood, genre classifier outputs) descriptors.

ETS / damped Holt–Winters. Exponential smoothing: the forecast is a weighted average of past values with weights that decrease geometrically, with a level, a trend that flattens out (damped) and a seasonal part.

Cross-validation, fold, stratified. See Sec. IV-D.

Fourier terms. $\sin(2\pi jt/m)$ and $\cos(2\pi jt/m)$ for $j = 1, \ldots, K$, where m is the season length (12 months, or 52.18 weeks) and K the number of harmonics; a regression on them draws any smooth yearly pattern.

Gini impurity. $1 - q^2 - (1-q)^2$ for a group with hit share q; a split is chosen to lower it the most.

Ljung–Box test. Tests whether the first L autocorrelations of a series are all zero (white noise).

Logistic regression. A linear model whose weighted sum of the inputs is turned into a probability by $1/(1+e^{-z})$; C is the inverse strength of the L2 penalty on the weights.

k-NN. Predicts the majority answer of the k most similar training songs.

MFCC. Mel-frequency cepstral coefficients: a compact description (13 or 20 numbers per frame) of the shape of the sound spectrum on a perceptual frequency scale; standard for timbre.

Million Song Dataset (MSD). A public collection of metadata and audio features for one million songs.

Onset envelope, tempogram. The onset envelope measures, frame by frame, how much the spectrum suddenly increases (note attacks); the tempogram is its local autocorrelation, showing which beat periods are present.

One-hot encoding. A category with m values becomes m yes/no inputs, exactly one of which is 1.

Out-of-fold. A value for a training song produced by a model that did not see that song (inner split).

PCA. Principal component analysis: new axes, combinations of the inputs, ordered by how much of the variance they explain.

Periodogram. The squared magnitude of the Fourier transform of a series: how much variance each period (cycle length) explains.

Permutation importance. Accuracy (or AUC) lost when one input is shuffled across the test songs.

RBF SVM. A support vector machine with a radial (Gaussian) kernel.

Seasonal naive. Forecast equal to the value observed one season (12 months) earlier.

Standardisation. Replacing each input by (value − mean) / standard deviation, the mean and standard deviation being computed on the training part.

Stemming. Reducing a word to its stem with fixed suffix rules (Porter stemmer: "causes" → "caus", "future" → "futur", while

"love" stays "love"); the musiXmatch release is stemmed this way.

STL; trend and seasonal strength. Seasonal–trend decomposition by local regression (Loess): series = trend + seasonal + residual. Trend strength is $\max(0, 1 - \mathrm{Var}(R)/\mathrm{Var}(T + R))$ and seasonal strength the same with S in place of T (0 = none, 1 = everything).

Spectral centroid, bandwidth, roll-off, flatness, contrast. Centre of mass of the spectrum; its spread; the frequency below which 85 % of the energy lies; how noise-like it is; the difference between peaks and valleys in each frequency band.

Track URI. Spotify's unique identifier of a track (`spotify:track:...`).

TreeSHAP, $|\phi|$. For a tree model, ϕ is how much an input moves one prediction away from the average prediction (SHAP value); the mean of $|\phi|$ over songs ranks the inputs.

Wide-and-deep. A neural network that combines a linear part on the raw inputs with a deep part (used by Zangerle et al.).

XGBoost. Gradient-boosted trees: trees are added one by one, each correcting the errors of the previous ones.

Appendix Q
Questions a reader may ask

Is artist history not cheating? No information from the song's own period or later is used (Fig. 4), so it is available on release day. What inflates its gain on the Hit Predictor is the construction of the negatives (artists of hits are excluded), which is why we report the year-matched ISMIR gain (+2.6 pts) as the transferable number.

Why does the paper say the gain holds for unseen artists but not for new artists? The two words mean different things (end of Sec. IV-E): an unseen artist is absent from the training folds but may have earlier hits; a new artist has none, so $h(s) = 0$.

Why Random Forest and not XGBoost? XGBoost is 1.3 points better under cross-validation but ties on the canonical split ($p = 0.87$); the frozen Random Forest keeps our numbers comparable with the published configuration, and the gain of our features is the same for all five classifiers (Table X).

Why no deep learning? The question is which inputs carry signal, at equal model. A learned audio representation could capture what the descriptors miss, and is future work.

Why reject the song title when it is the strongest feature? A title does not make a song a hit. Its gain comes from formatting ("- Remastered", "feat.") that only non-hit titles carry, and from genre and language words; once both are controlled, what remains (+1.9 pts, $p = 0.002$) is only indicative, below the Bonferroni threshold, and is itself mostly genre and language vocabulary (Appendix J).

Why does release year alone score below chance (0.448) in ISMIR protocol B? Every hit has a non-hit from the same year, so the year carries no information, and a Random Forest on a single input with many distinct values fits noise in the training folds; with exactly balanced years, what it learns for a year is then, on average, the opposite of the test fold's labels for that year, which pushes it below 0.5.

Why is FFT rejected although its gain is significant on the whole benchmark? Because it vanishes when hits and non-hits come from the same genres (Table XVI): spectral descriptors recognise the niche genres from which the benchmark draws its non-hits.

Why are the canonical numbers lower than the cross-validated ones? The canonical split trains on 20,000 songs, cross-validation on 32,448; with more training data every configuration is about one point better, and the differences between configurations are the same.

Why are lyrics kept but not genre, if lyrics also partly reflect genre? Spotify genre tags describe the artist as known today, so they are not known on release day. The words of a song are known on release day. On ISMIR the negatives are random catalogue songs, so the difference in themes between chart songs and the rest is real rather than built into the dataset; we nevertheless state that lyrics measure mainstream-ness more than the appeal of one song (Sec. V-G).